

Department of Computer Science and Engineering

Course Title: Software Engineering

Code: CSE412

Section: 2

Project Proposal

Project Title: LocalLinker

Submitted To:

Yasin Sazid

Lecture

Department of Computer Science & Engineering

Submitted by

Name	ID
Rafia Noor Nithin	2022-2-60-120
Swarna Rani Dey	2022-1-60-340
Tofayel Ahmed Talukder	2022-1-60-055
Nelufa Yeasmin	2020-1-60-236

Date of Submission : 15.05.2025

1. Introduction

Finding trusted service providers can be difficult. Many people struggle with unreliable workers, unclear pricing, and security concerns. Locallinker solves this by offering a secure platform where customers can easily find, book, and pay for verified professionals.

1.1.1 Project Overview

Locallinker is an AI-powered Local Service Management System designed to connect customers with verified service providers for home repairs, cleaning, electrical work, and other professional services. The platform ensures a secure and seamless service booking experience by integrating real-time communication, AI-driven recommendations, secure payments, and identity verification. It aims to enhance trust, convenience, and efficiency in local service transactions while providing a structured and fraud-free marketplace.

Requirements as a user story

Customers in need of reliable service providers can register and authenticate themselves through a secure sign-up and login process. Once logged in, they can browse available service providers based on location, ratings, price, and category. After selecting a provider, they can confirm bookings, choose a suitable date and time, and proceed with secure payment using multiple options, including online transactions and OTP verification. Real-time tracking and communication allow customers to interact with providers before service delivery. Once the service is completed, customers can rate their experience, leave feedback, and report any issues, ensuring accountability and continuous platform improvement.

Service providers must register and submit verification documents before listing their services. After admin approval, they can manage bookings, update availability, and receive real-time notifications. Providers benefit from an AI-powered recommendation system that increases visibility based on performance, customer ratings, and demand trends. They can chat with customers, respond to inquiries, and track their service history for better management.

Administrators are responsible for verifying service providers, approving or rejecting applications, and ensuring compliance with platform policies. They monitor system activity, detect fraudulent behavior, and take necessary actions to maintain platform integrity. By generating insights and reports on service trends, user behavior, and platform performance, administrators ensure a secure and efficient service environment.

Locallinker provides a trustworthy, scalable, and user-friendly service booking experience, benefiting customers, service providers, and administrators alike.

1.1.2 List of Requirements

To ensure a seamless, secure, and efficient service booking experience, the system must fulfill both functional and non-functional requirements. These requirements are derived from stakeholder needs, ensuring the platform is user-friendly, scalable, and secure. Additionally, advanced features, known as wow factors, enhance user engagement and platform reliability.

Functional Requirements (FRs)

1. **User Management** – Secure registration, login, and authentication.
2. **Service Discovery** – Search, filter, and browse service providers.
3. **Booking System** – Schedule, modify, and cancel bookings.
4. **Payment System** – Secure transactions with multiple options and OTP verification.
5. **Real-time Communication** – Integrated chat and notifications.
6. **Admin Oversight** – Provider verification, fraud detection, and compliance monitoring.

Non-Functional Requirements (NFRs)

1. **Security & Privacy** – Data encryption, secure authentication, and fraud detection.
2. **Performance & Scalability** – Optimized for high concurrent user handling and low response time.
3. **Usability & Accessibility** – Intuitive UI, responsive design, and ease of use.
4. **Reliability & Availability** – High uptime, fault tolerance, and continuous system monitoring.

Extraordinary Requirements (Wow Factors)

- **AI-driven Smart Matching** – Recommends service providers based on past ratings and preferences.
- **Live Tracking of Service Providers** – Ensures customer safety and transparency.
- **Real-Life Payment System** – Integrated with real-world payment gateways for seamless transactions.

1.1.3 Technology Stack:

LocalLinker will be developed using a modern, scalable technology stack that ensures real-time communication, AI-driven recommendations, and secure payment processing. The selected technologies focus on high performance, security, and future scalability. This stack supports modular development, making the platform easy to maintain and upgrade as needed.

The following technologies are proposed:

Category	Technologies
Programming Languages	JavaScript, Python
Frontend Frameworks/Technologies	React.js, Next.js, HTML, CSS
Backend Frameworks	Node.js with Express.js, PHP
Database	MongoDB (NoSQL) or MySQL (SQL)
Authentication	JWT (JSON Web Token), OAuth 2.0
Real-time Communication	WebSocket (or Firebase)
AI Engine	Python libraries (Random Forest, XGBoost)
Deployment & DevOps	GitHub Actions (CI/CD), Docker (Containerization), Cloudflare Workers (Cloud Hosting)
Testing & Development Tools	Jest (Unit Testing), Postman (API Testing)

1.2.1 Top-Level Component Diagram

The Top-Level Component Diagram decomposes the LocalLinker system into major functional modules such as Authentication, Service Management, Booking Management, Payment Processing, Communication Services, and Admin Management. It visually illustrates how these components interact internally to provide complete system functionality.

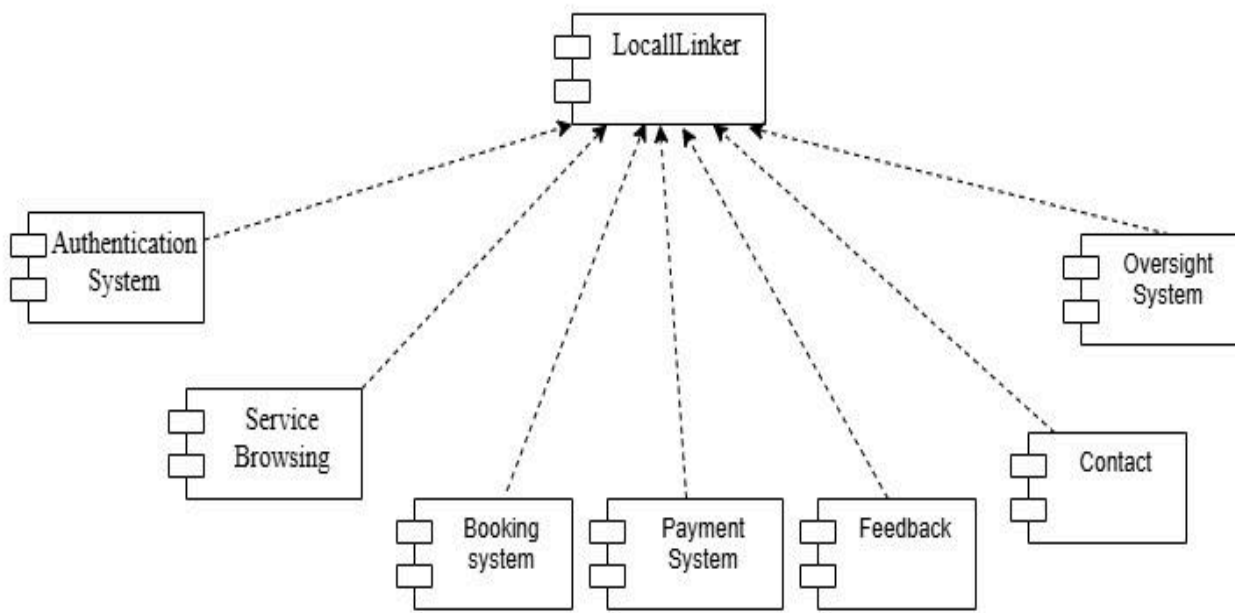


Figure 1:Top-Level Component Diagram for LocalLinker

1.2.2 Instantiation of Each Component with Component Elaboration

For each top-level component, a detailed elaboration is provided. This includes its internal structure, the responsibilities it handles, how it communicates with other components, and key data it manages. This breakdown ensures modularity, clarity, and ease of future maintenance or scaling.

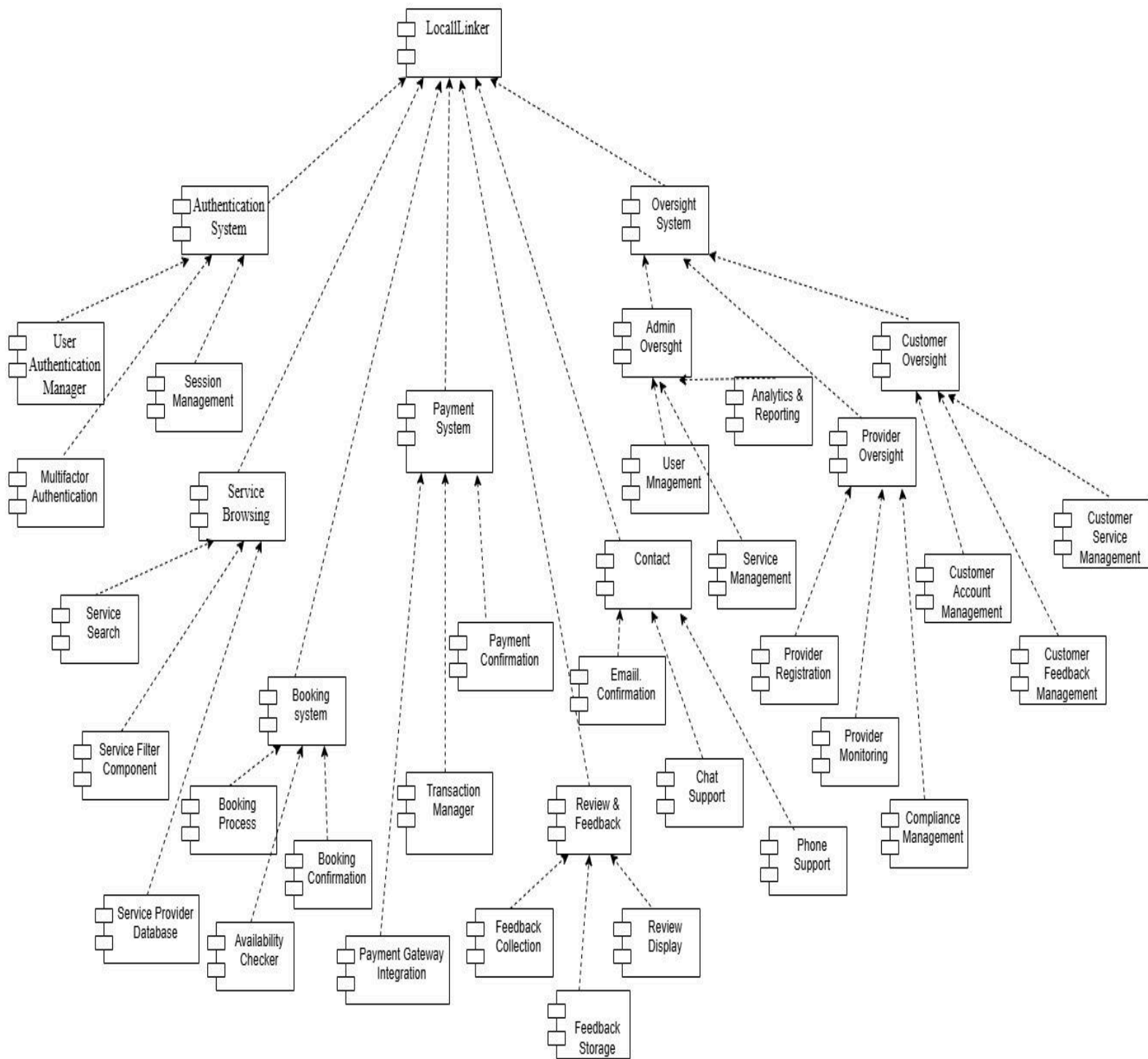


Figure 2: Instantiation of Each Component for LocalLinker

1.2.3 Component-Level Design

This section focuses on the detailed design of each major component within the LocalLinker system. It breaks down the internal structure of functional modules, defines key software classes and their interactions, and models how system data will be organized and managed. By elaborating component responsibilities, class structures, and database design, this section ensures that every part of the system is modular, scalable, and maintainable. The objective is to create a robust foundation that simplifies implementation, enhances system flexibility, and supports future upgrades or expansions.

1.2.4 Elaboration of Design Components

In this phase, each major component is elaborated through a structured approach that includes identifying subsystem responsibilities, modeling the interactions among different modules, and defining preliminary design classes. This detailed breakdown promotes modularity, reusability, and flexibility, ensuring that the LocalLinker system can evolve efficiently as future requirements grow or change.

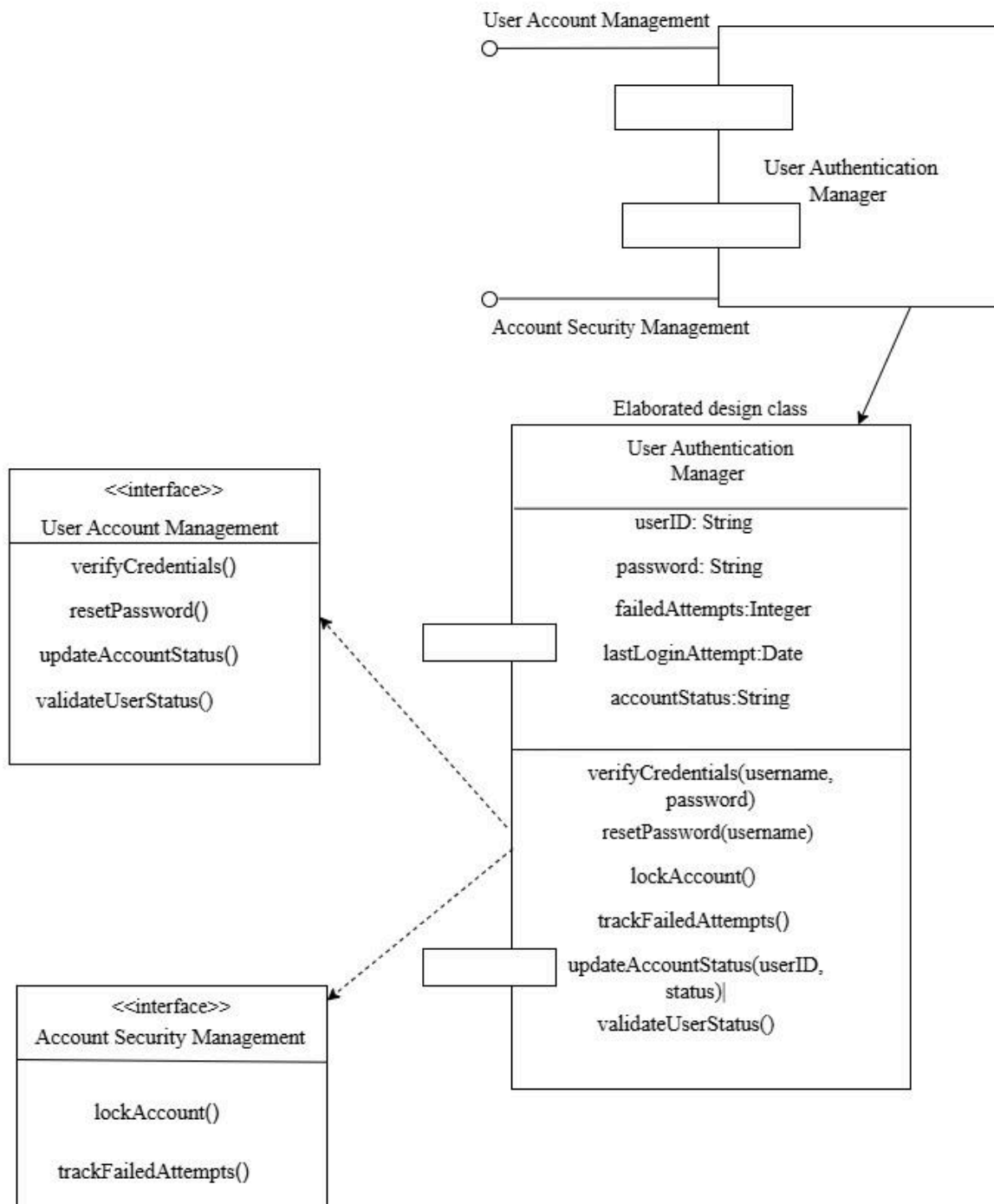


Figure 3:Elaboration of User Authentication Manager

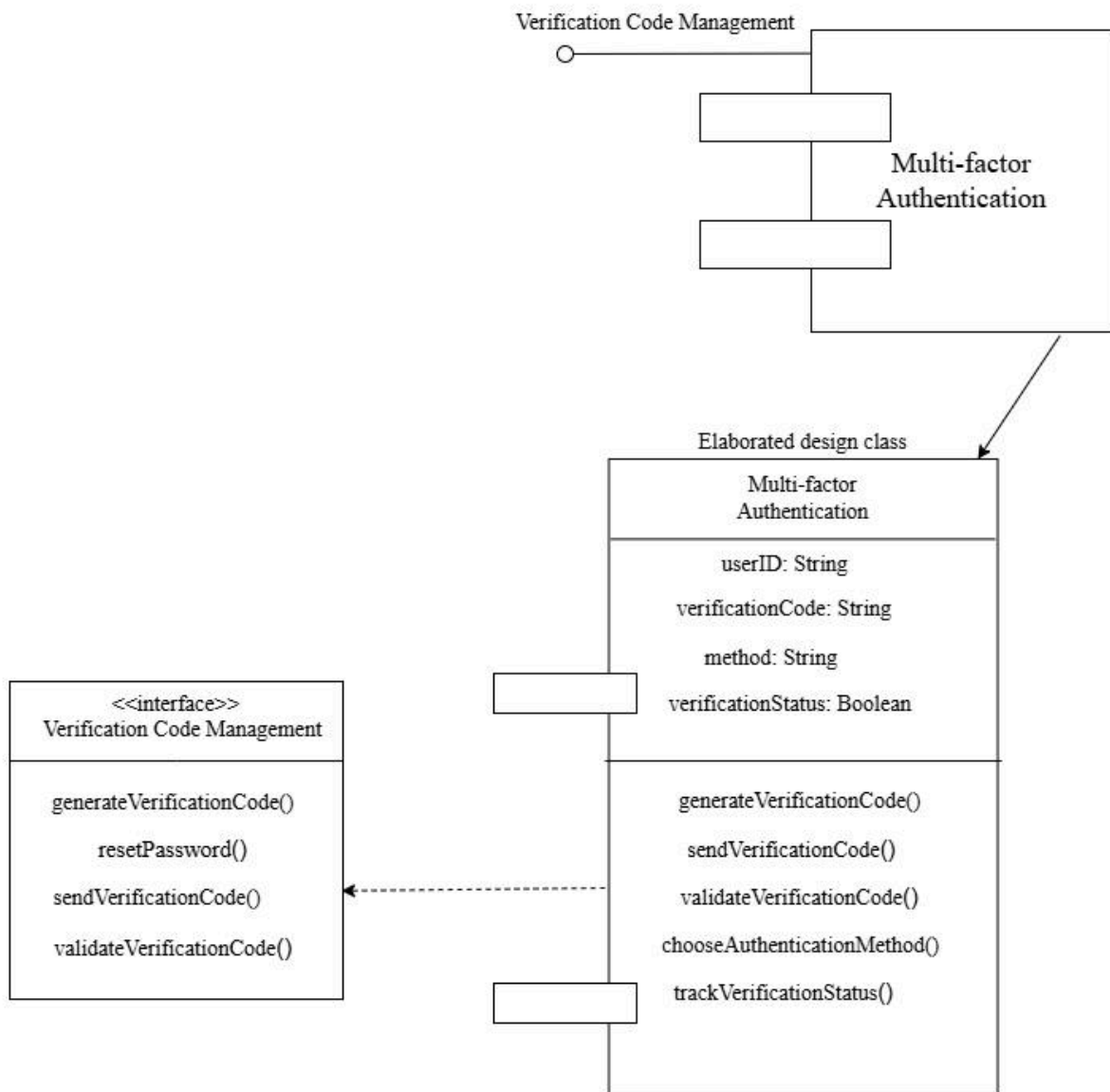


Figure 4:Elaboration of Multi-Factor Authentication

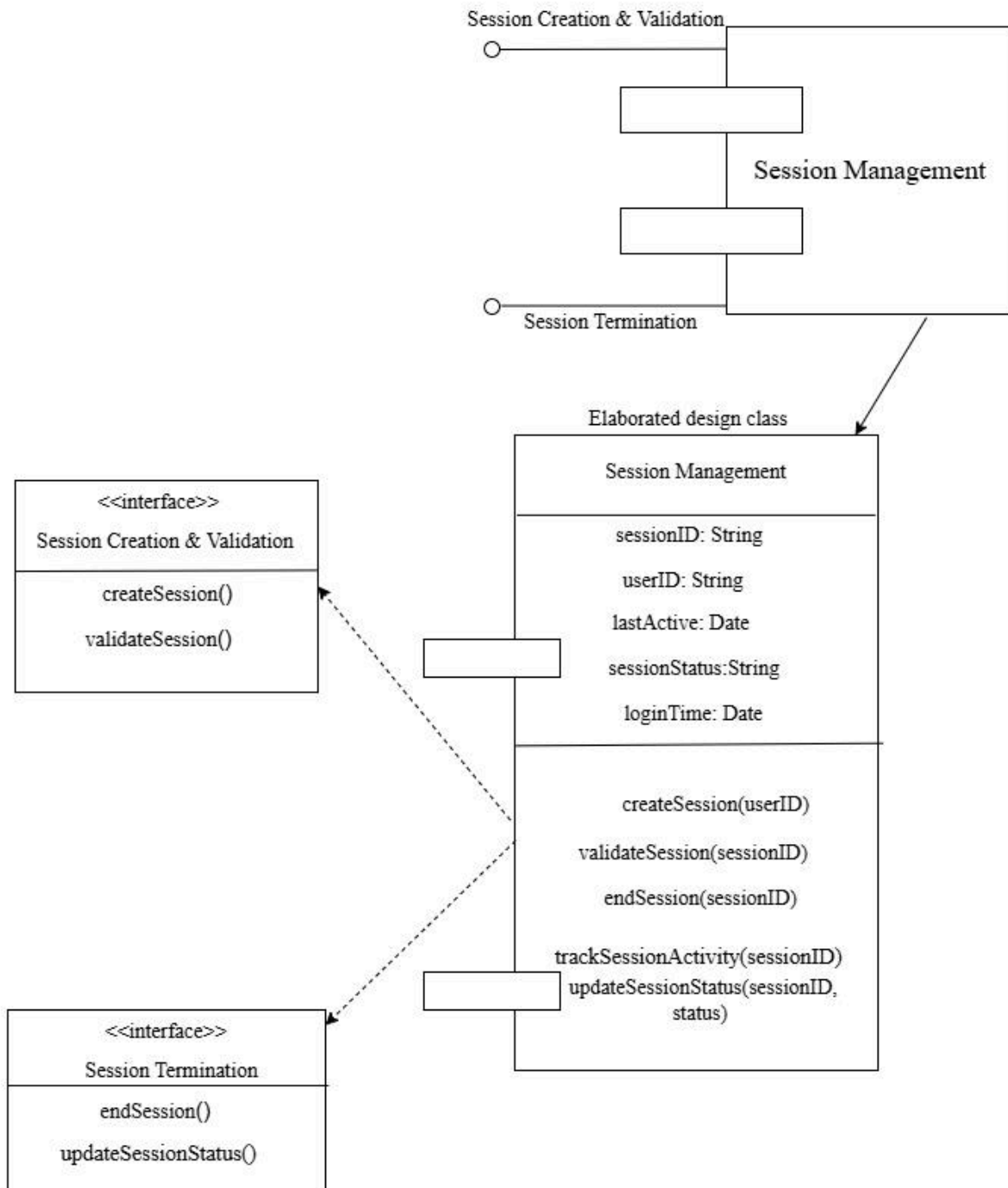


Figure 5:Elaboration of Session Management

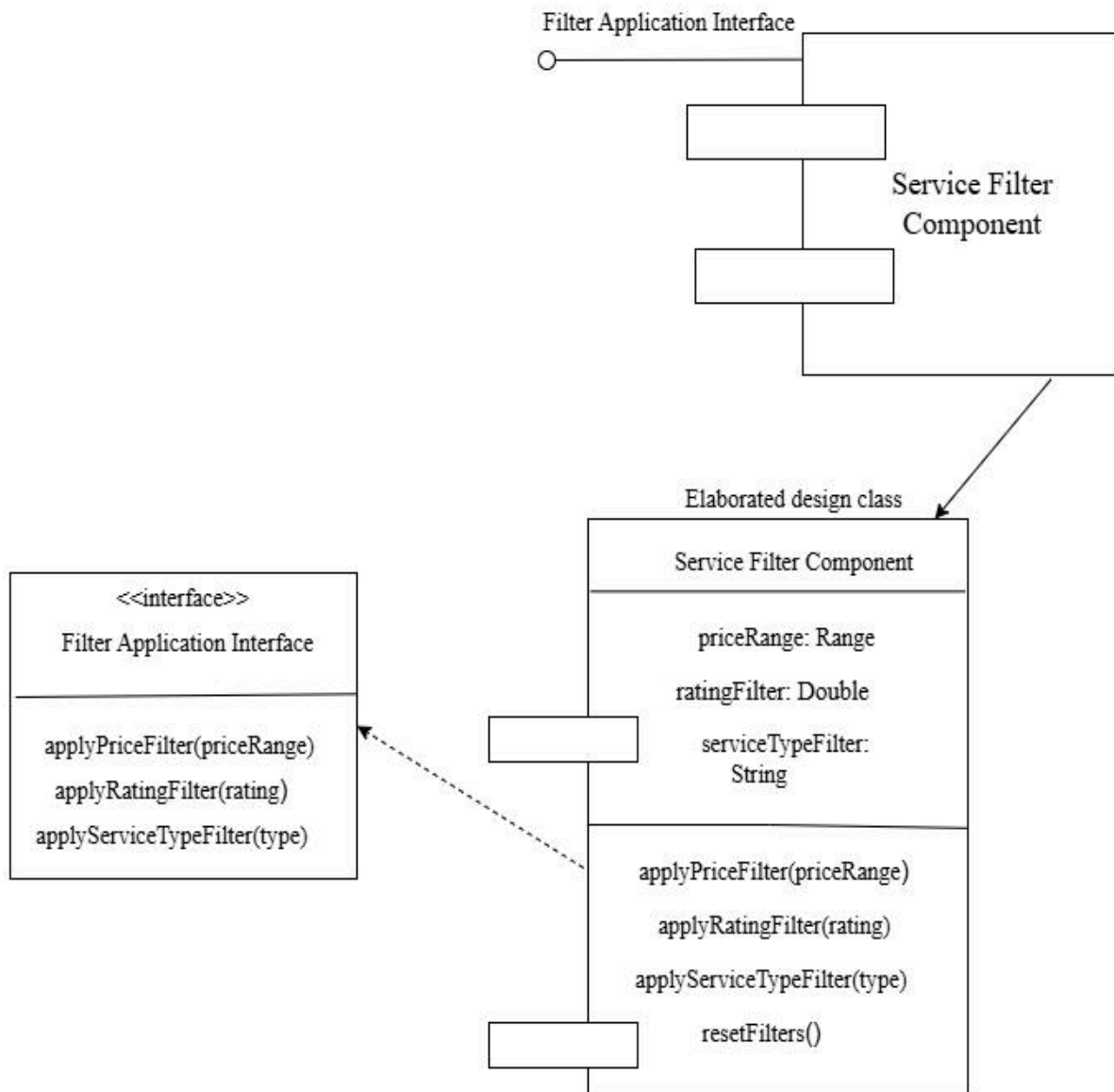


Figure 6:Elaboration of Service Filter Component

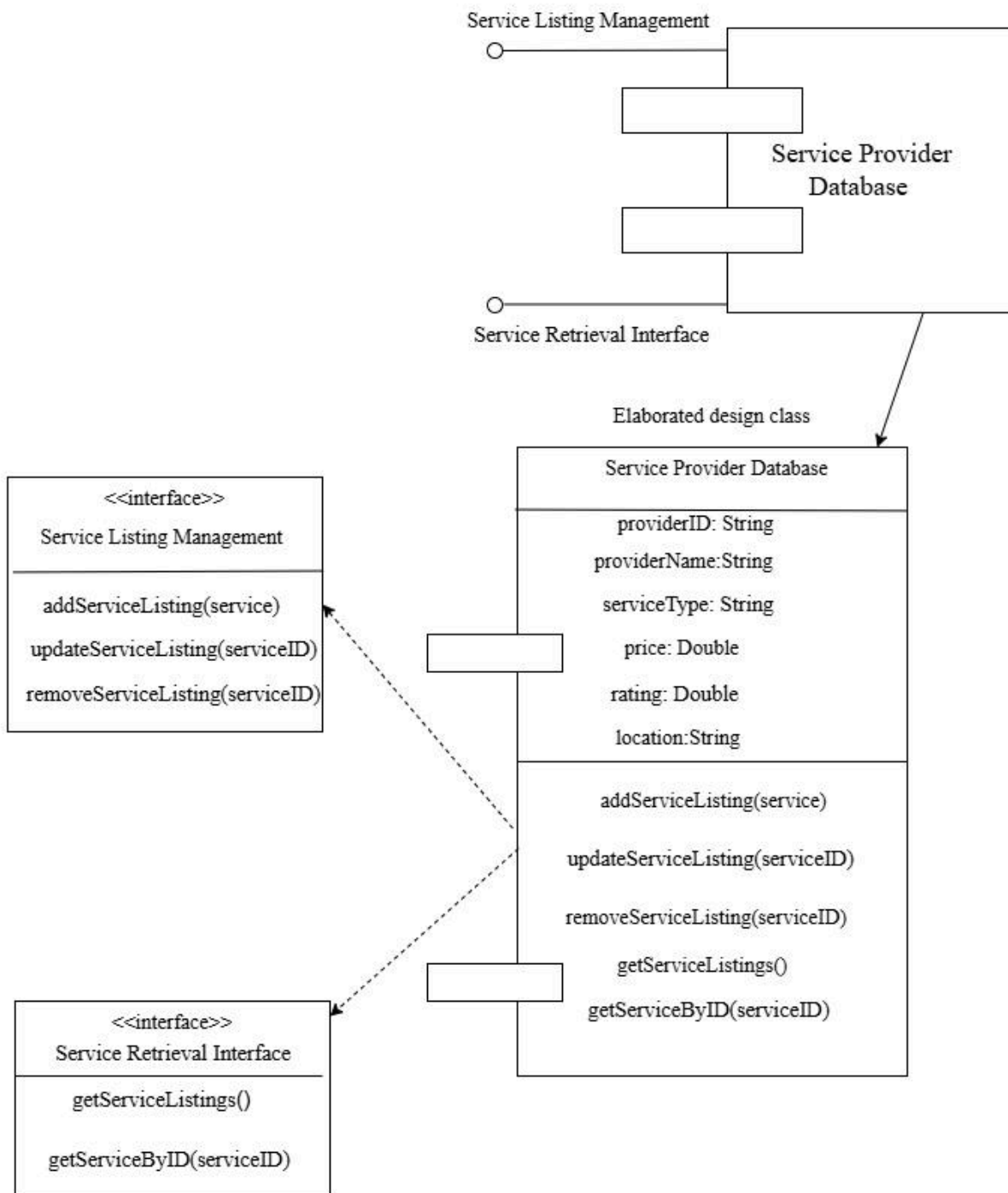


Figure 7:Elaboration of Service Provider Database

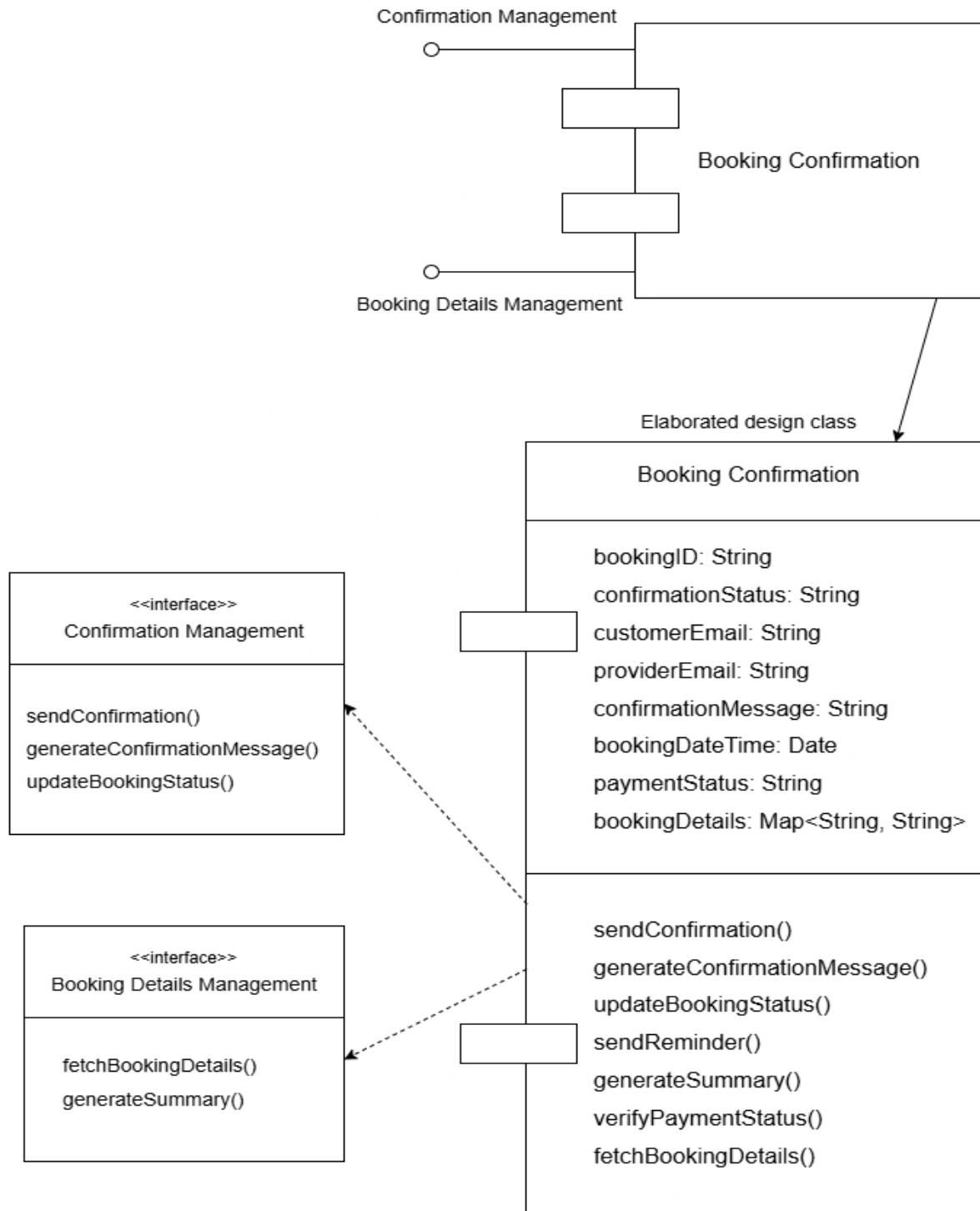


Figure 8:Elaboration of Booking Confirmation

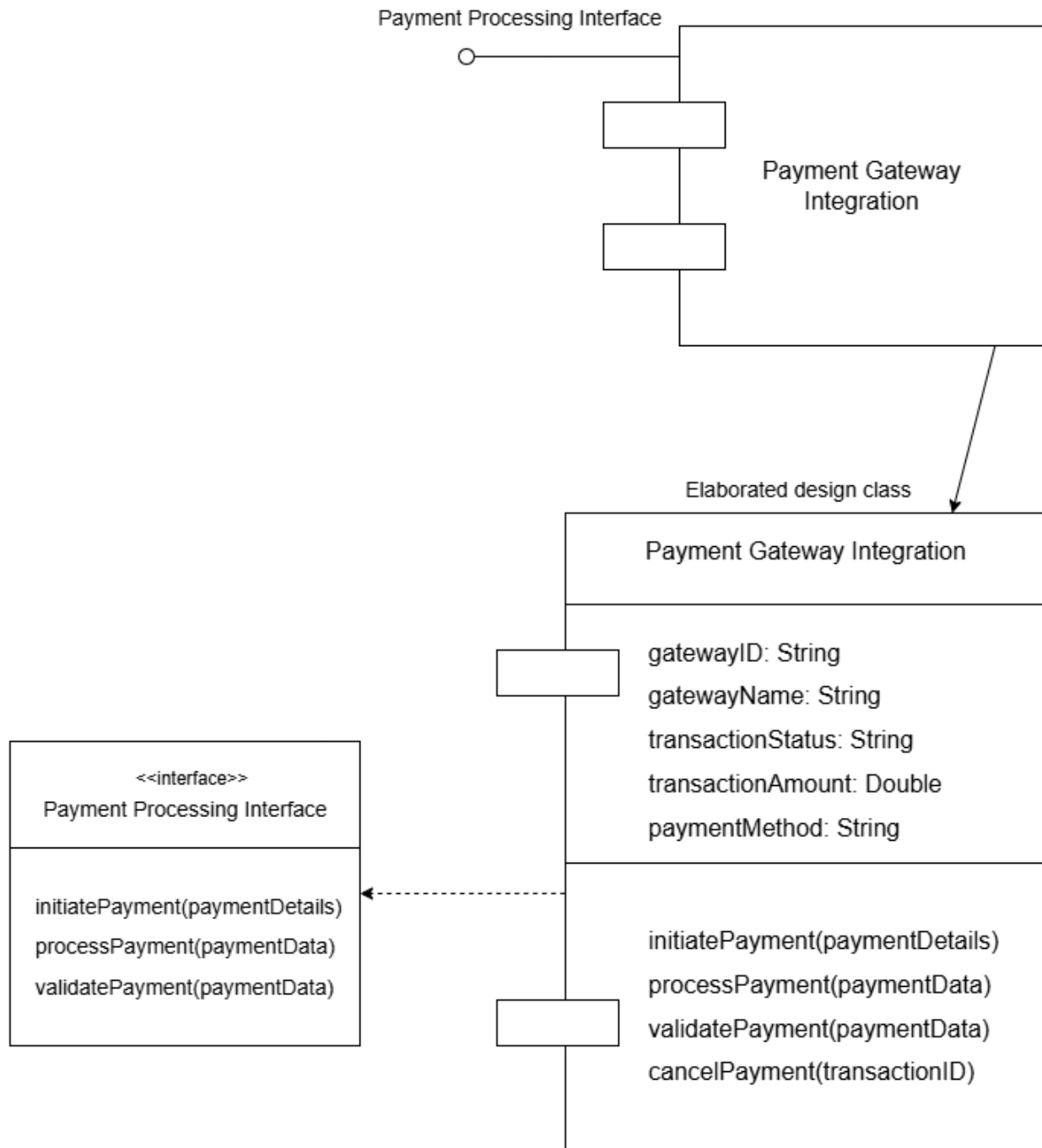


Figure 9: Elaboration of Payment Gateway Integration

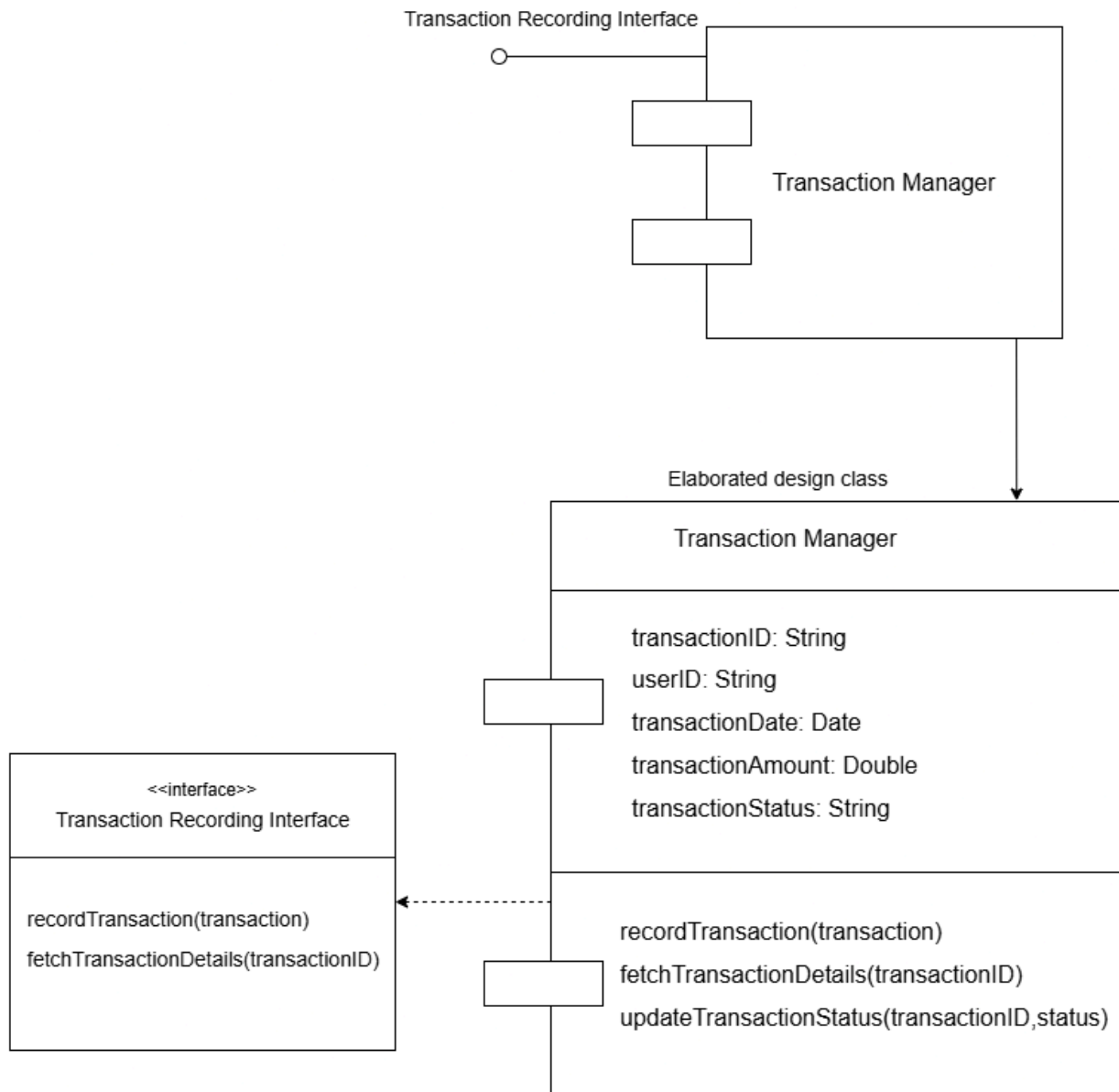


Figure 10: Elaboration of Transaction Manager

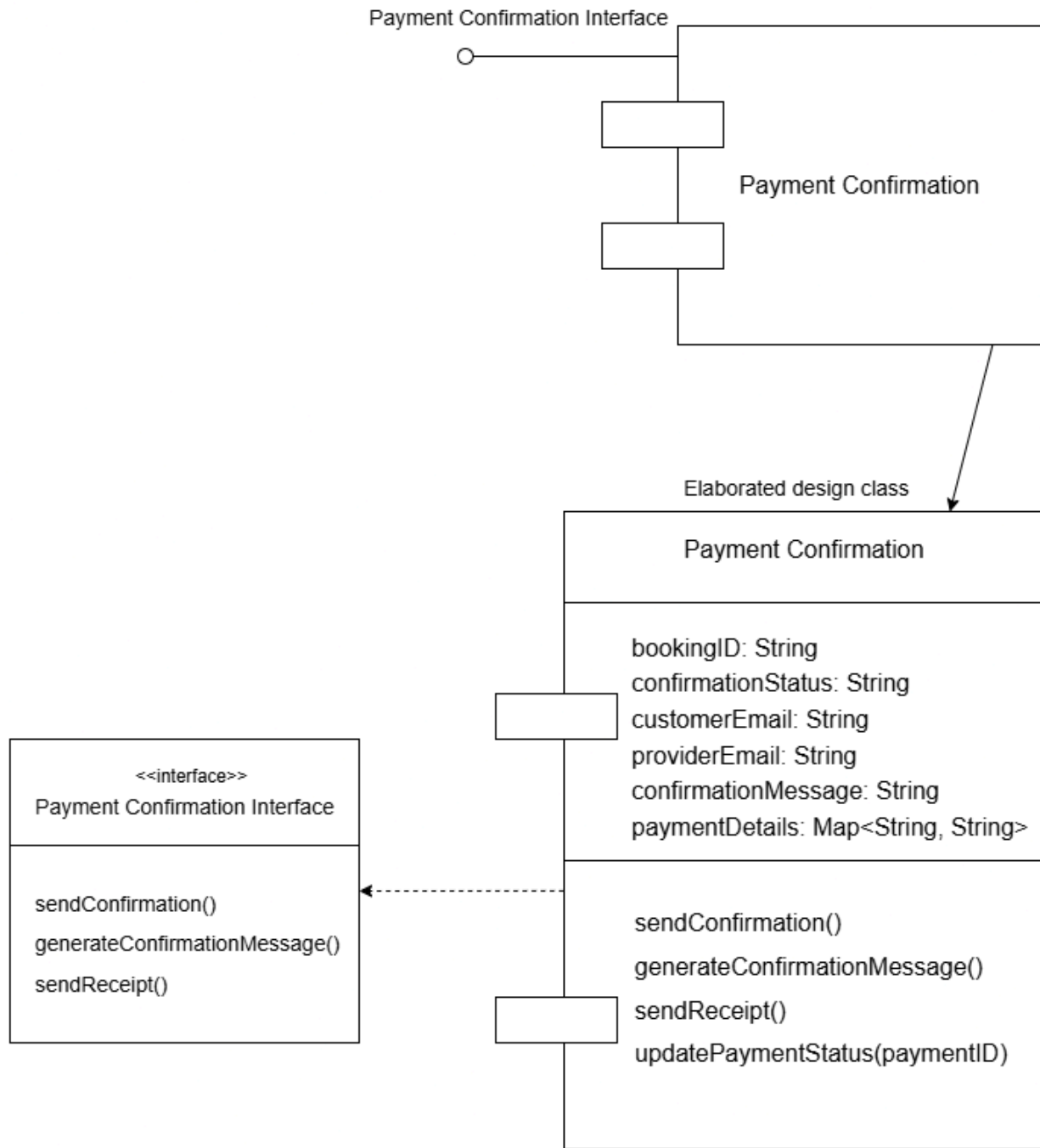


Figure 11: Elaboration of Payment Confirmation

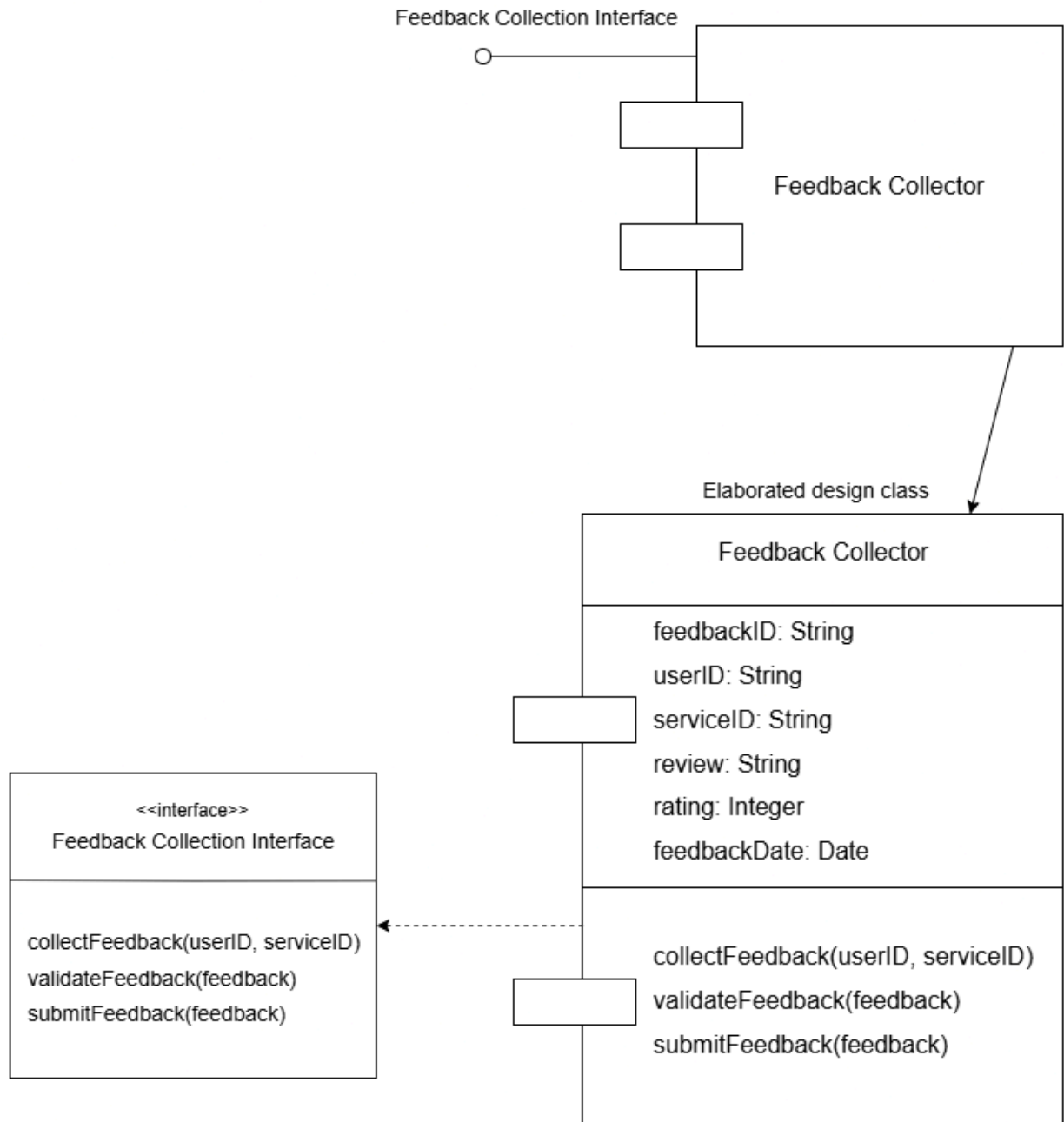


Figure 12: Elaboration of Feedback Collector

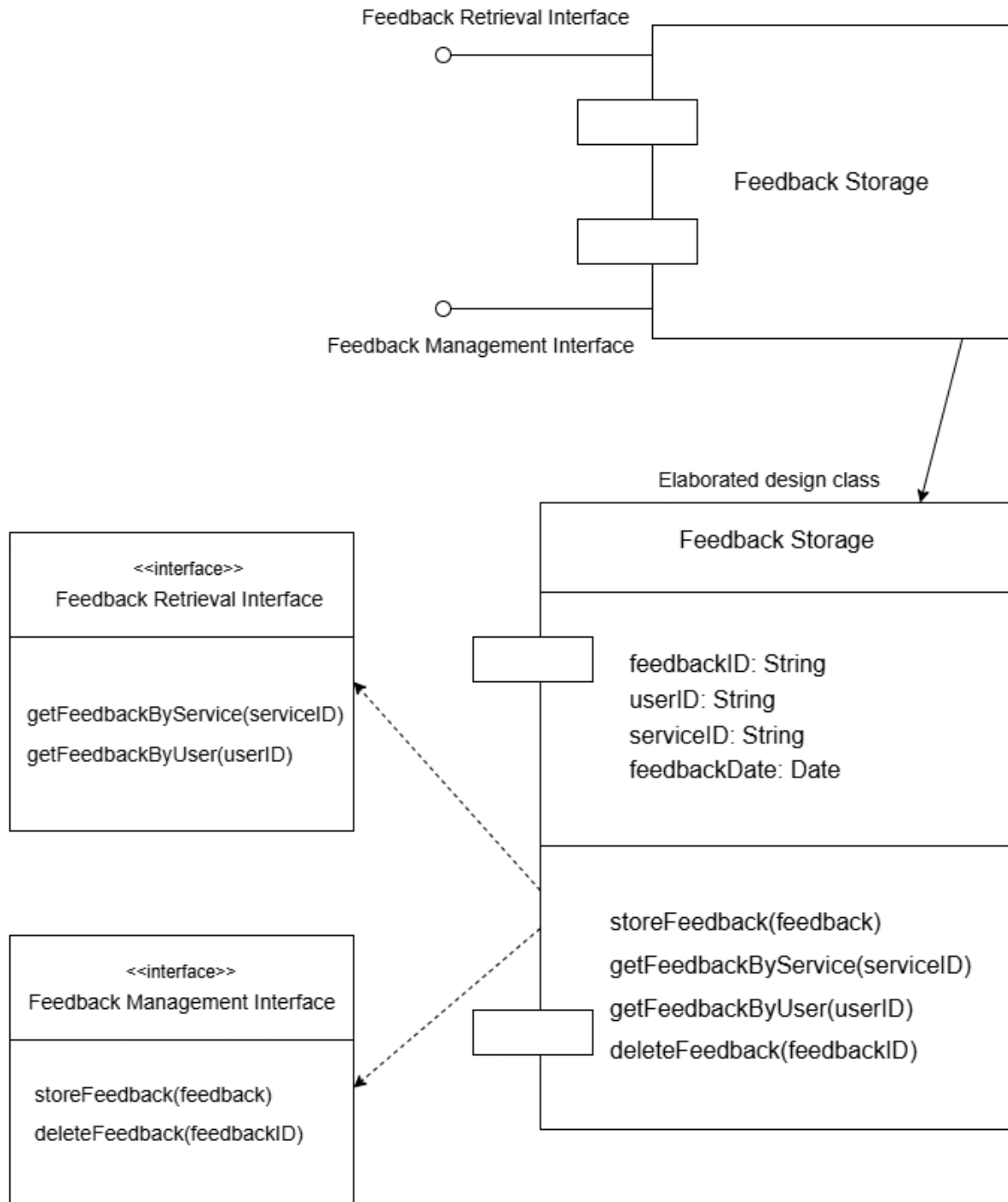


Figure 13: Elaboration of Feedback Storage

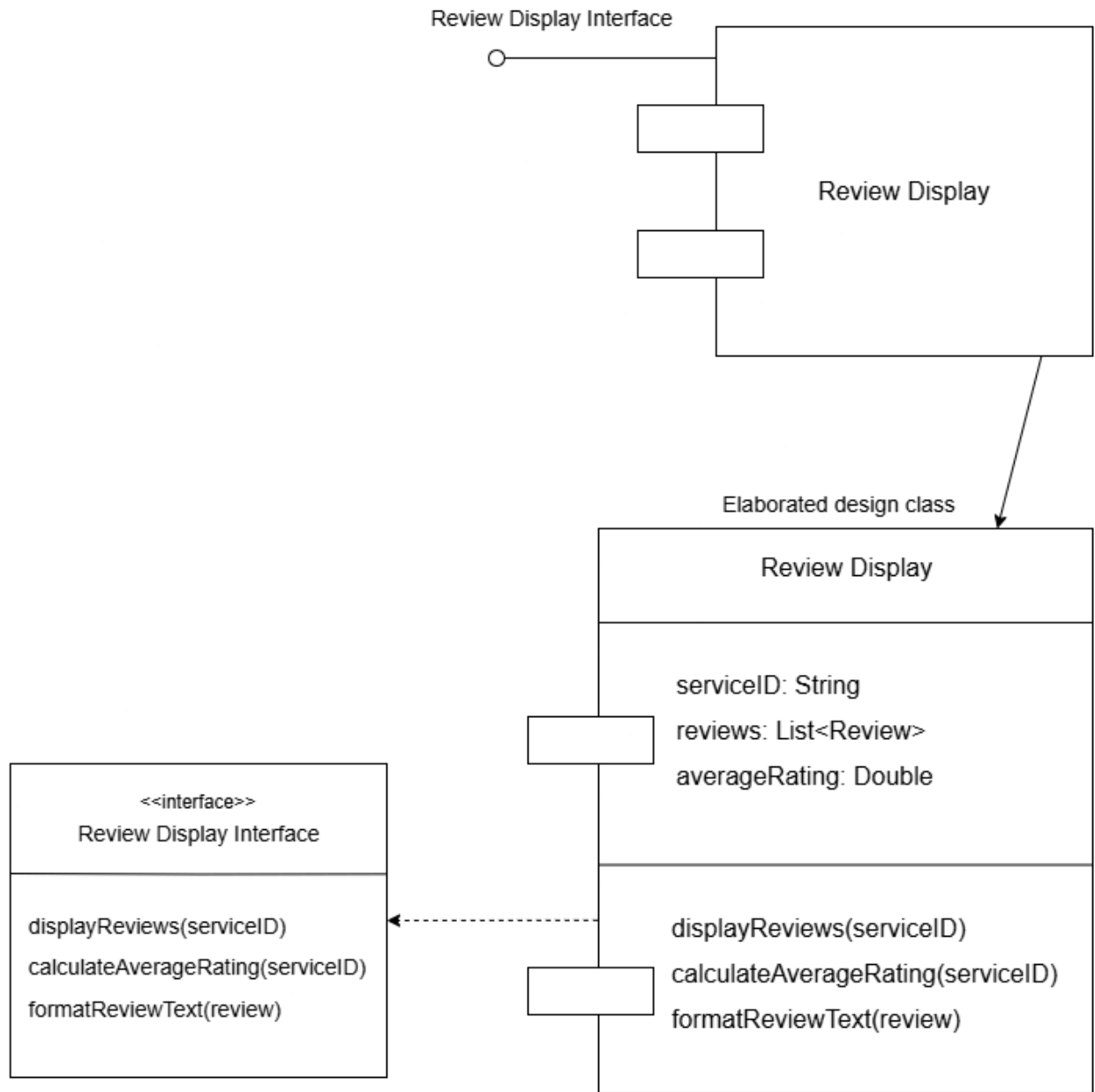


Figure 14: Elaboration of Review Display

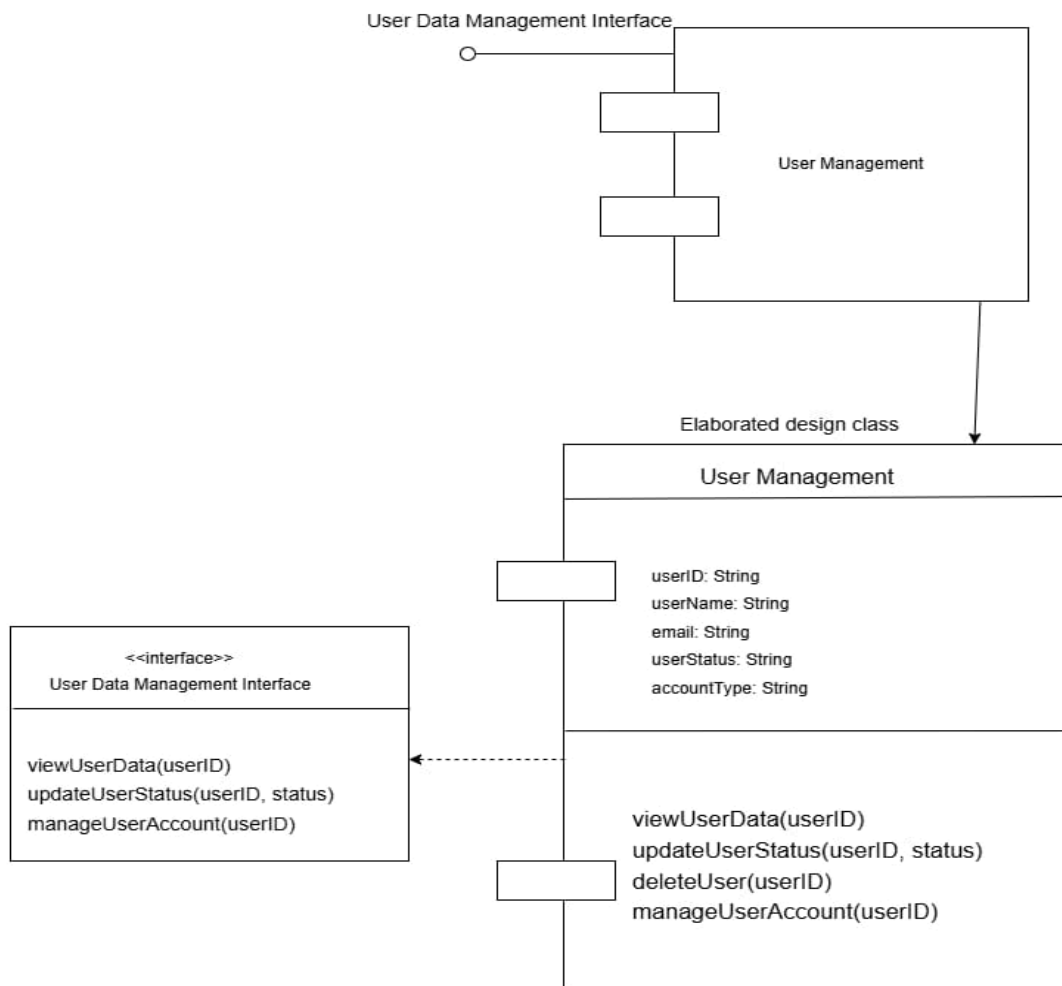


Figure 15: Elaboration of User Management

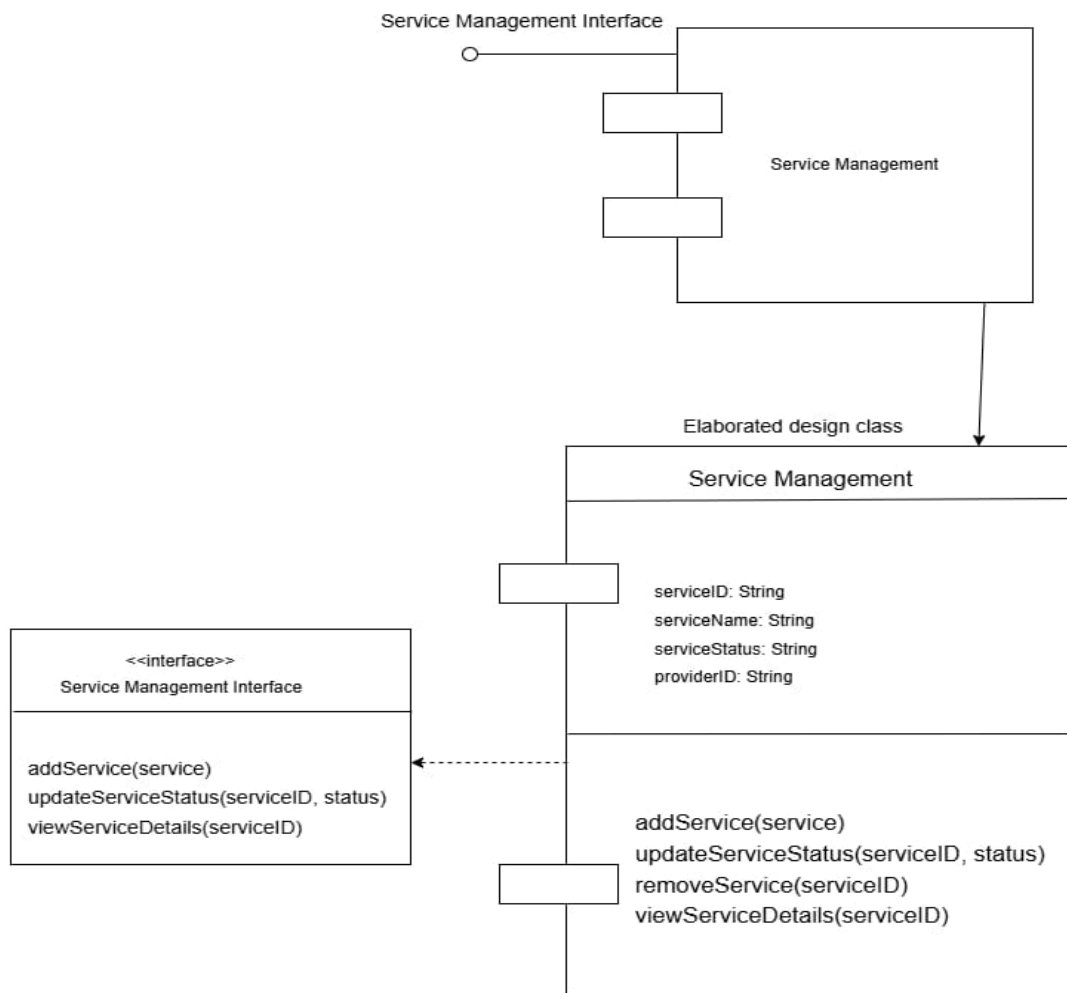


Figure 16: Elaboration of Service Management

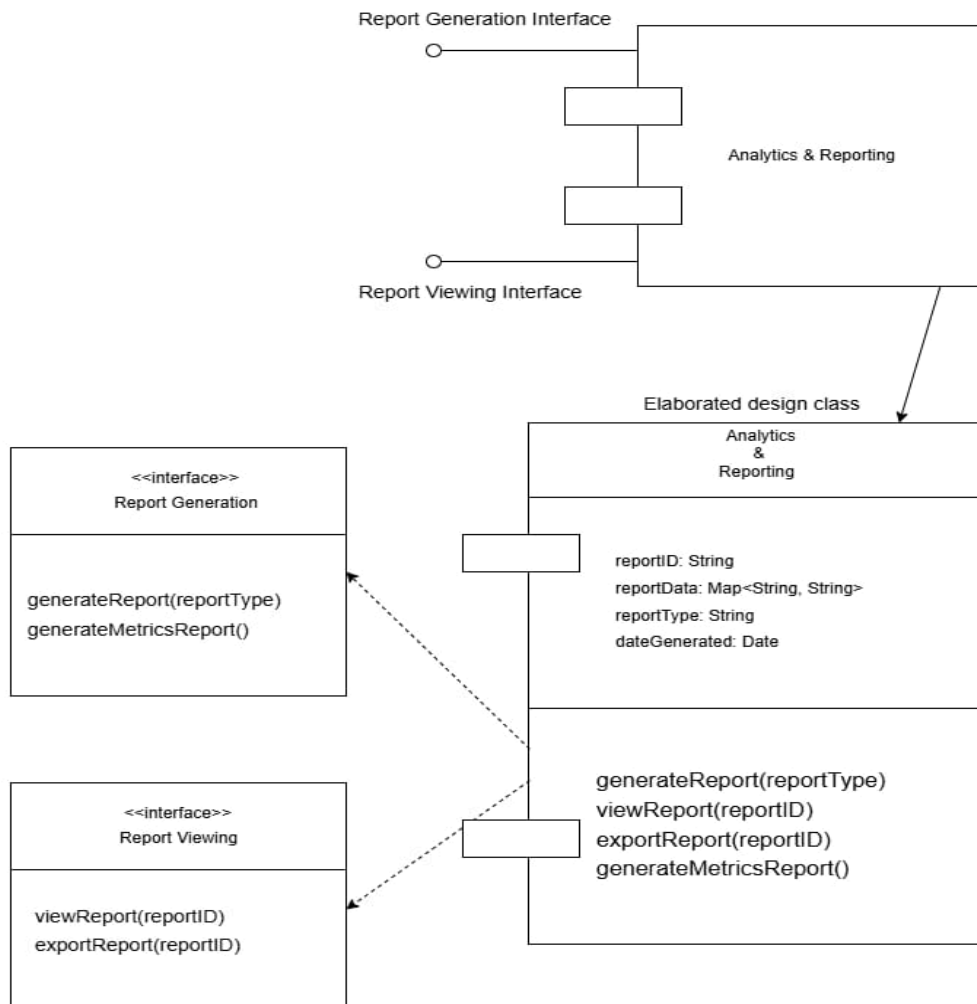


Figure 17: Elaboration of Analytics & Reporting

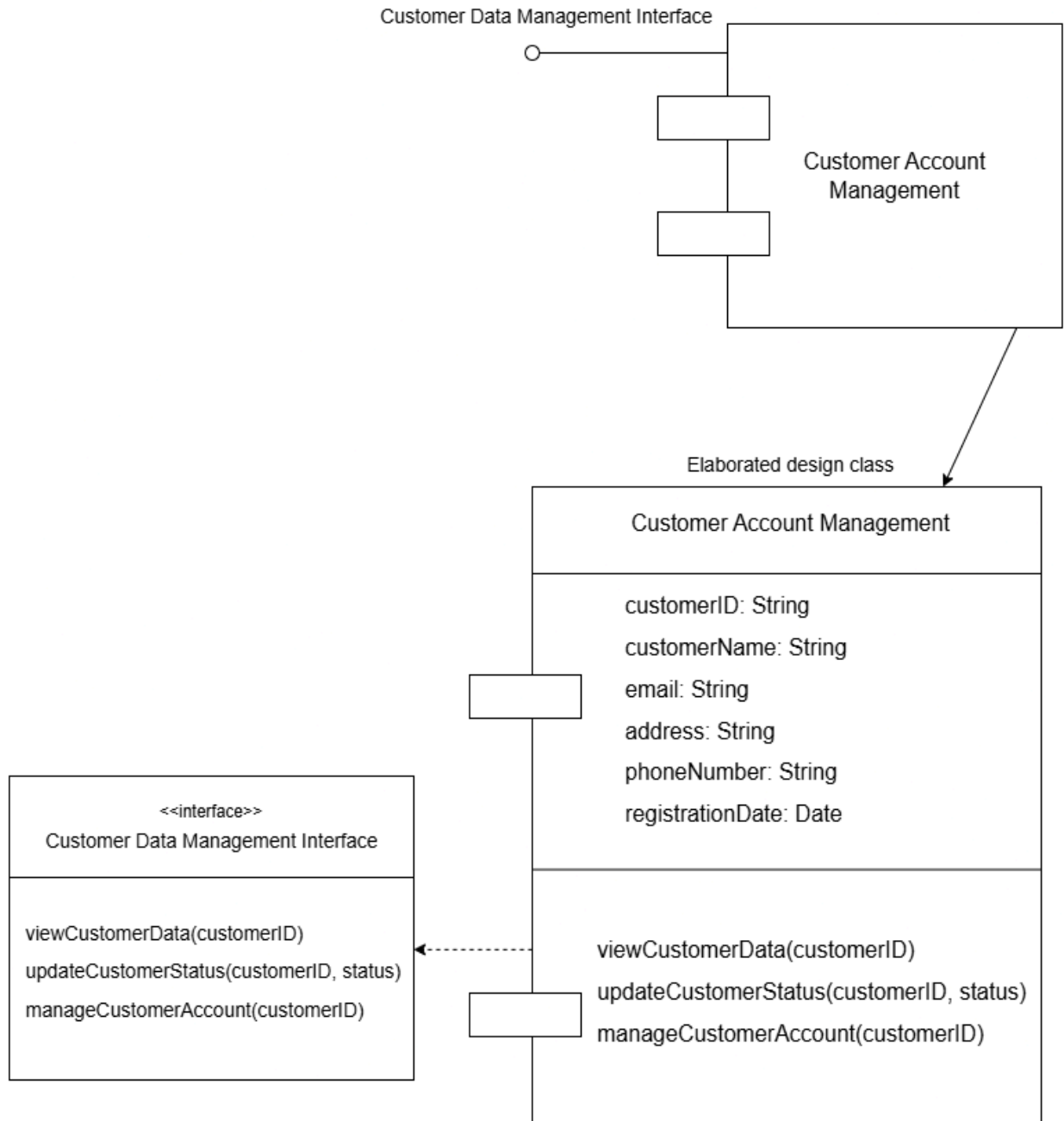


Figure 18: Elaboration of Customer Account Management

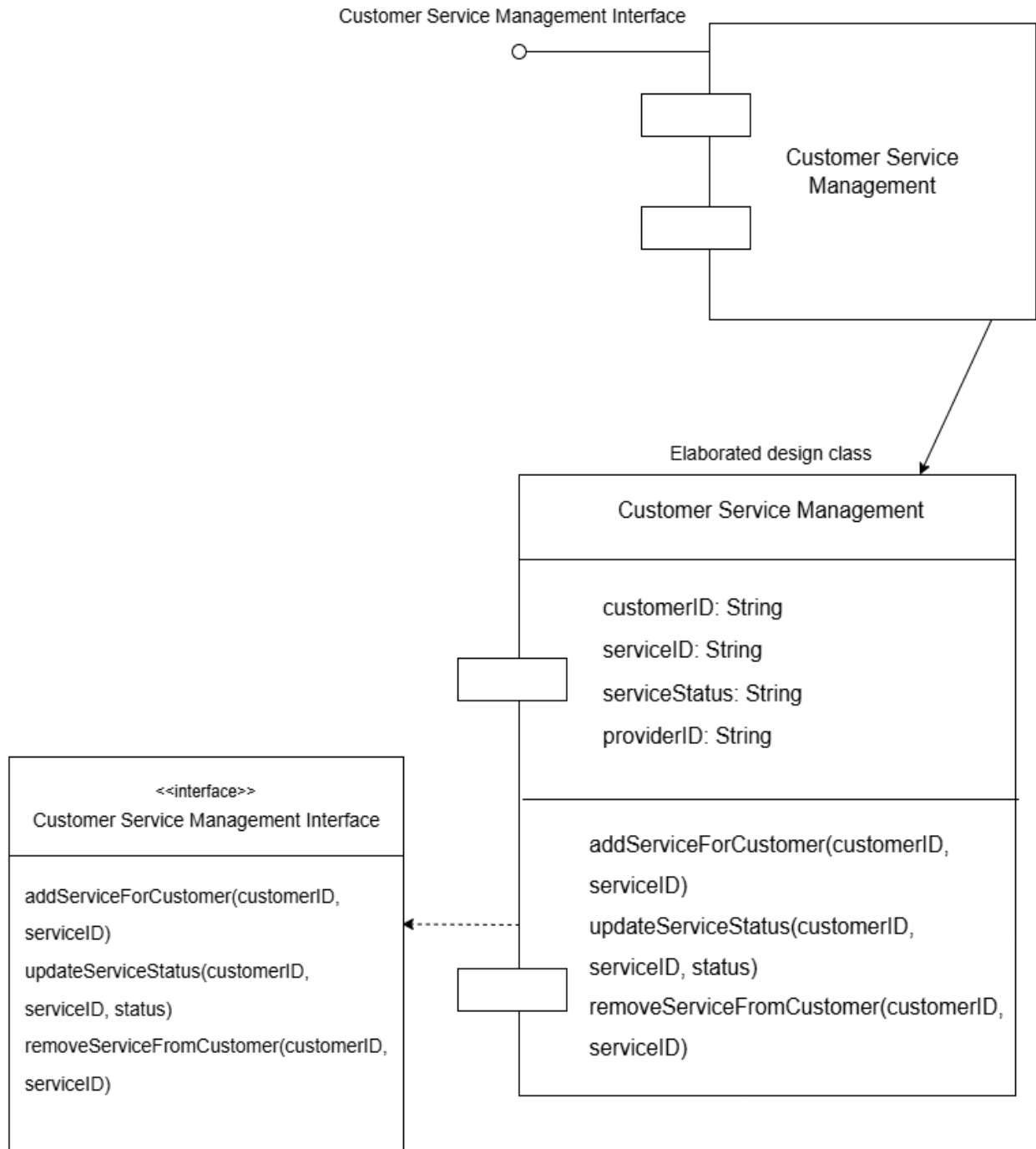


Figure 19: Elaboration of Customer Service Management

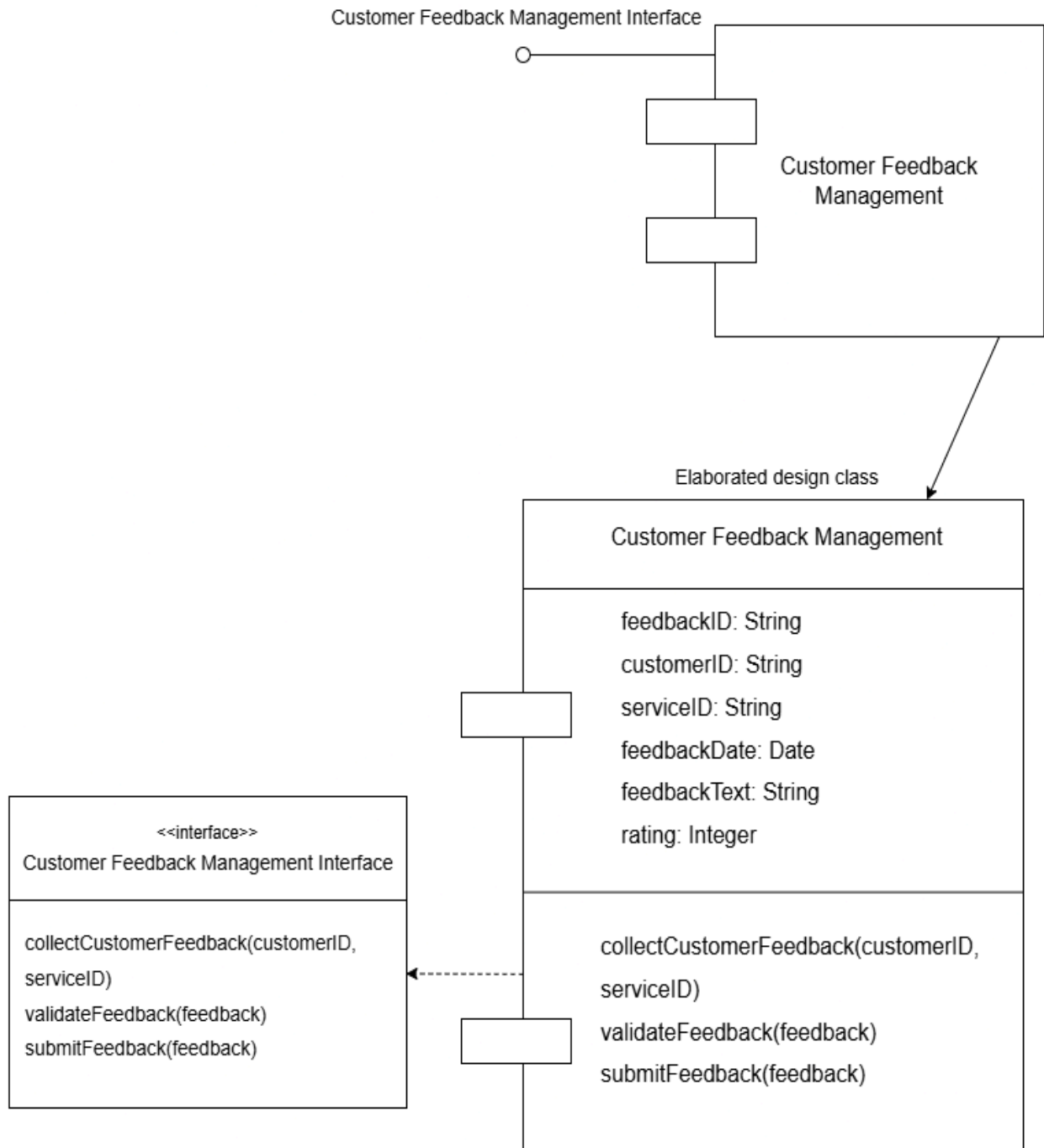


Figure 20: Elaboration of Customer Feedback Management

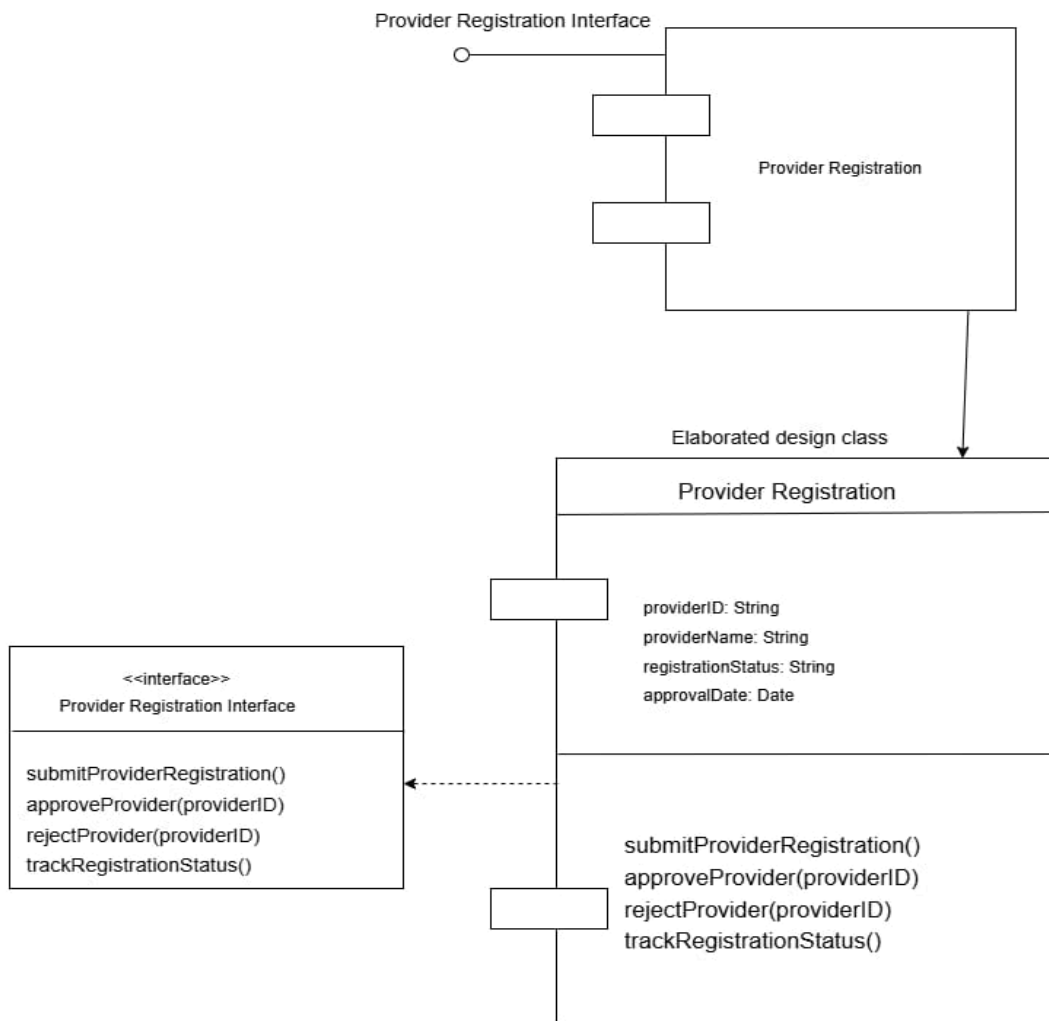


Figure 21: Elaboration of Provider Registration

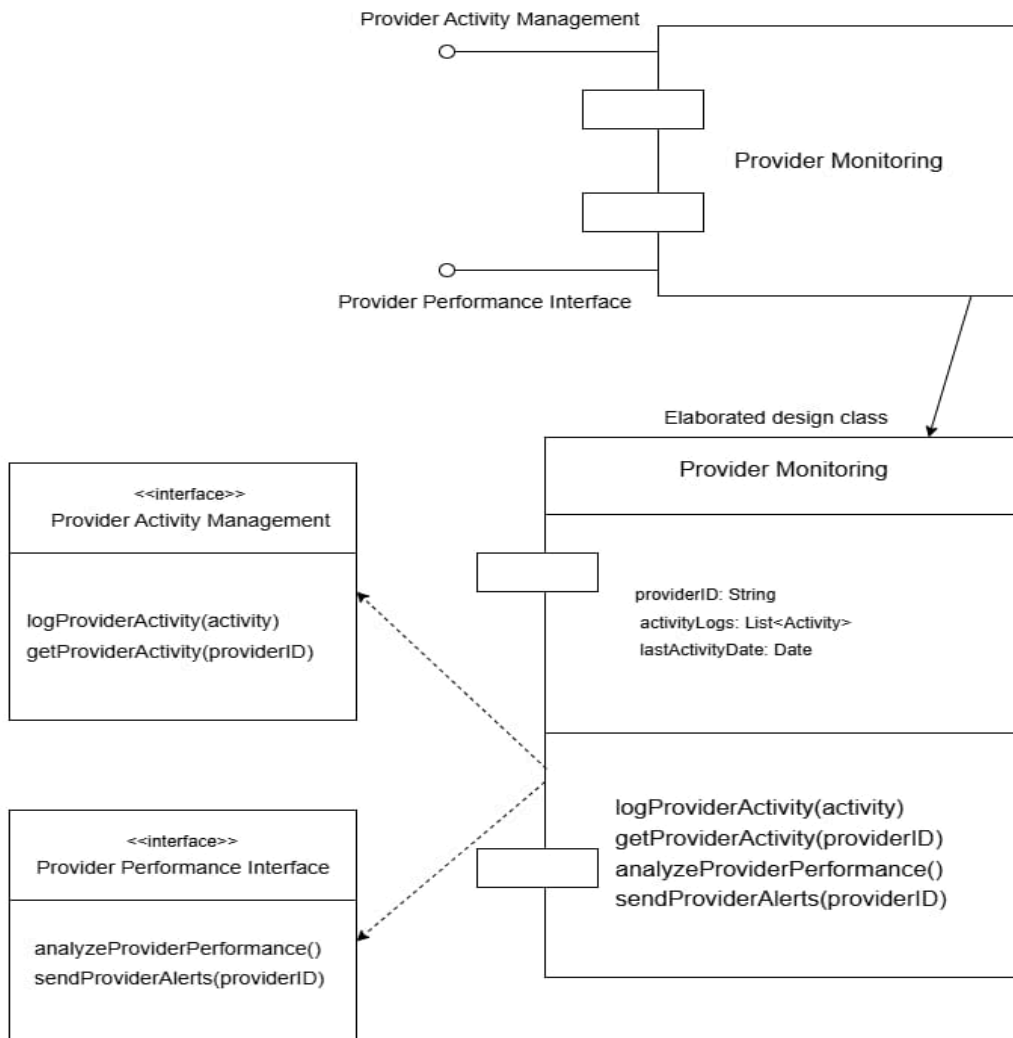


Figure 22: Elaboration of Provider Monitoring

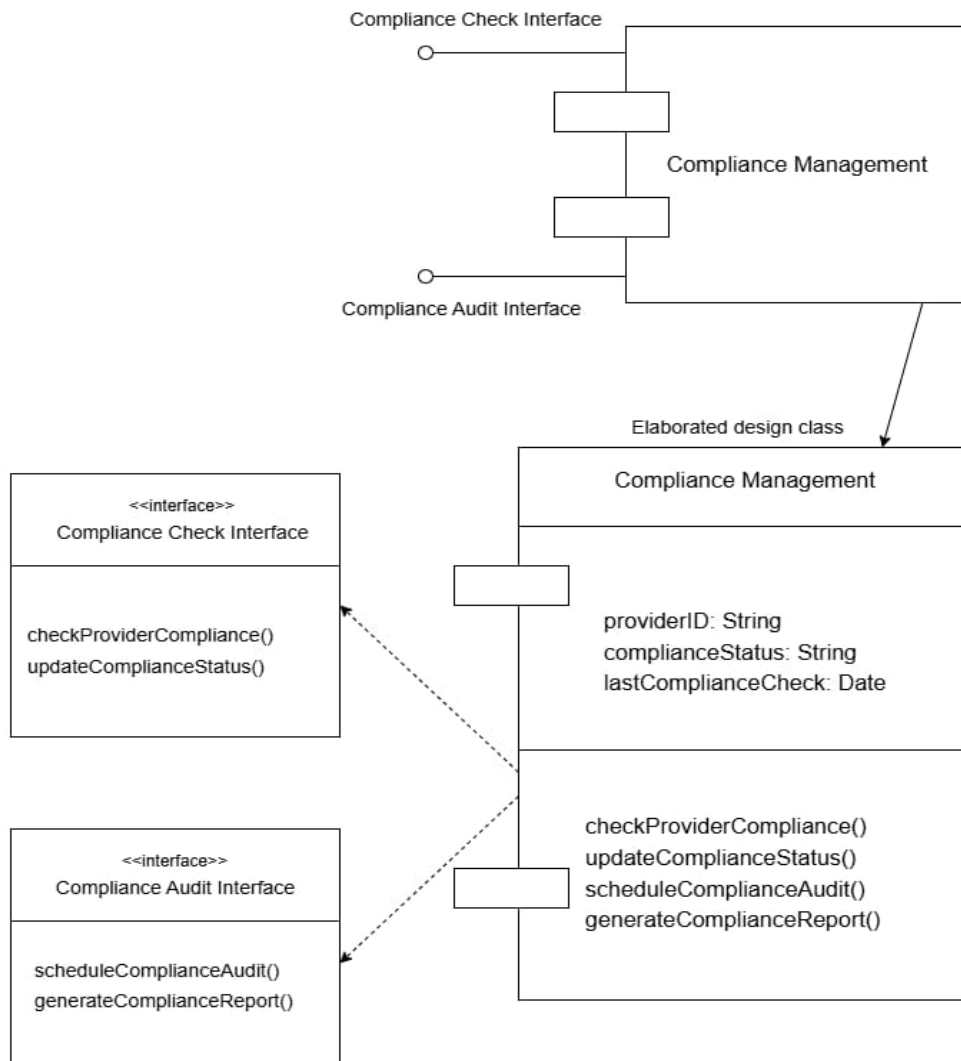


Figure 23: Elaboration of Compliance Management

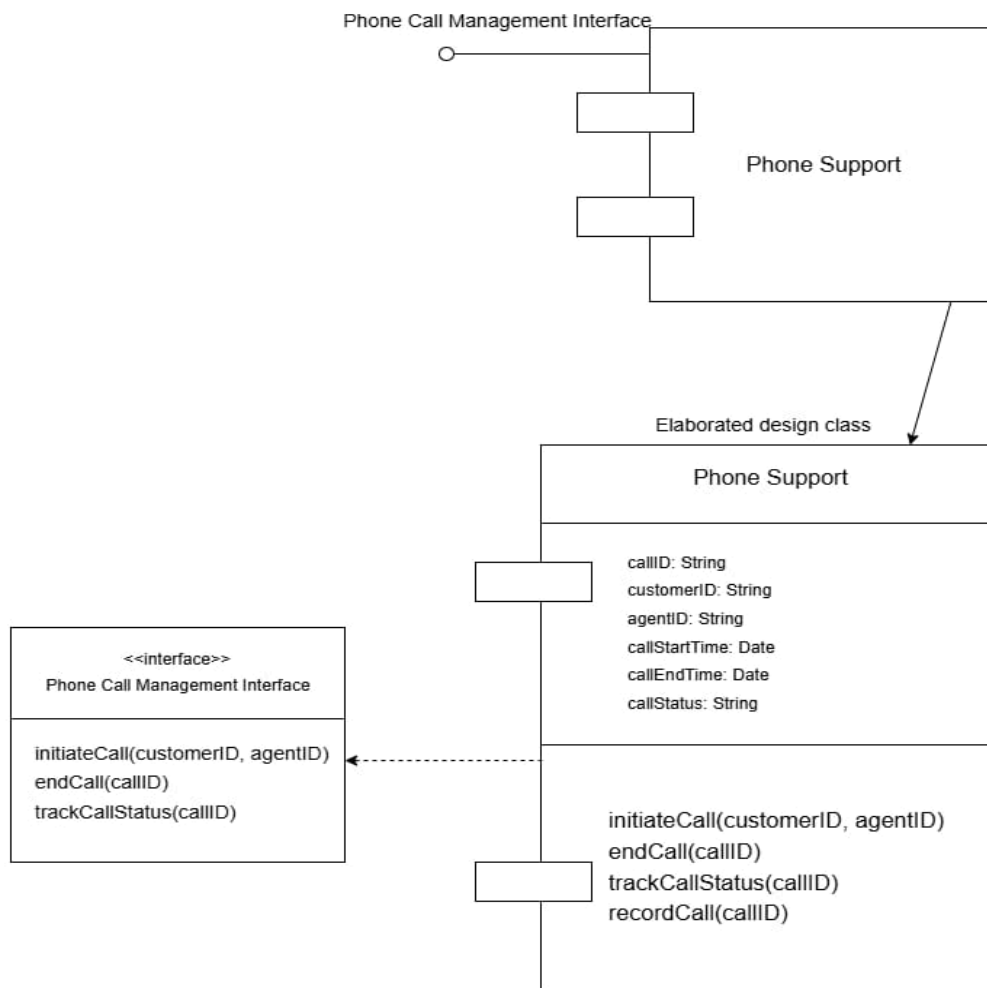


Figure 24: Elaboration of Phone Support

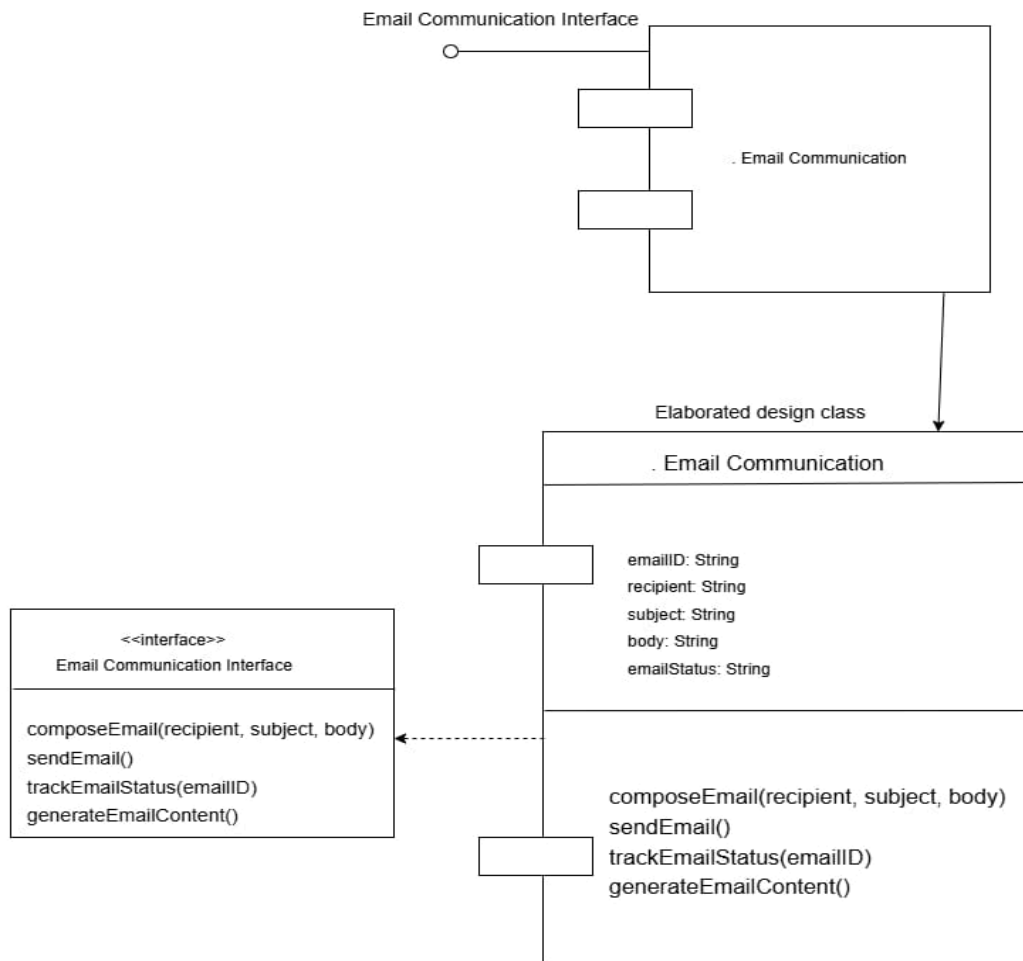


Figure 25: Elaboration of Email Communication

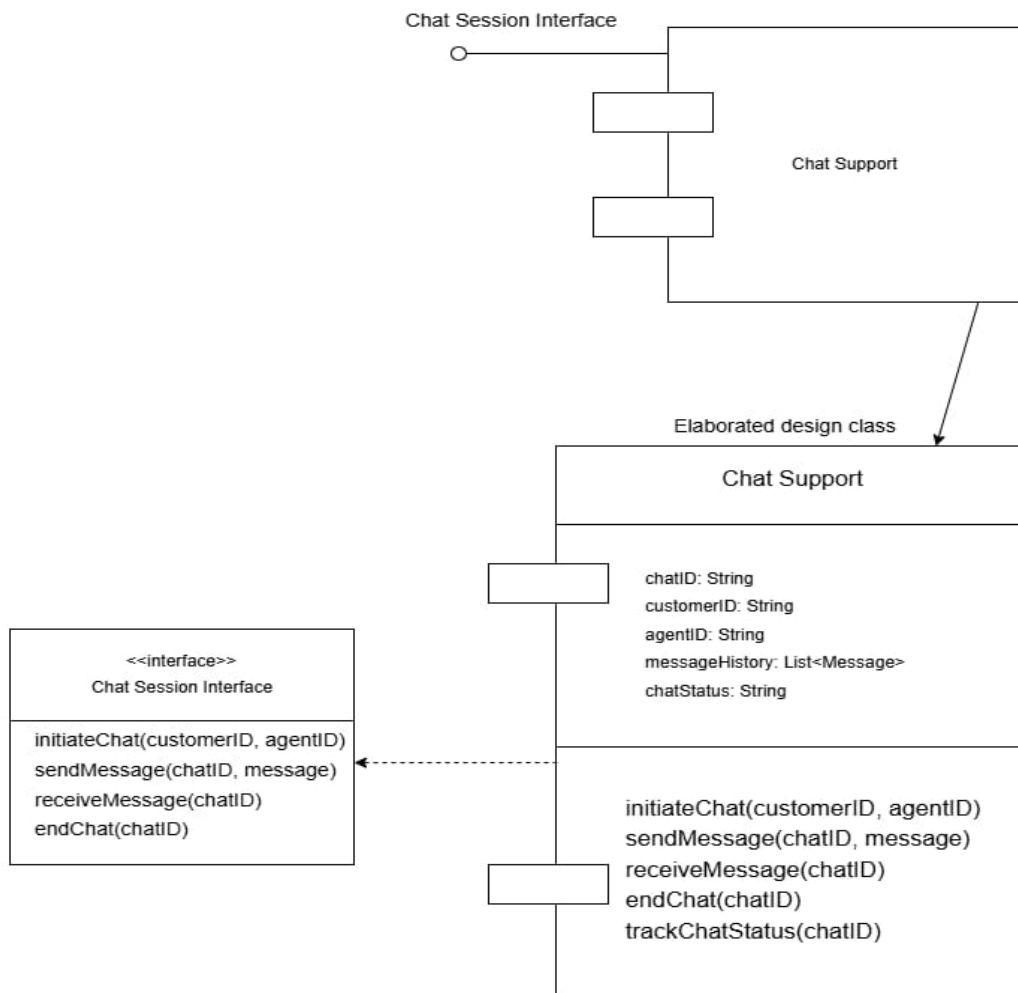


Figure 26: Elaboration of Chat Support

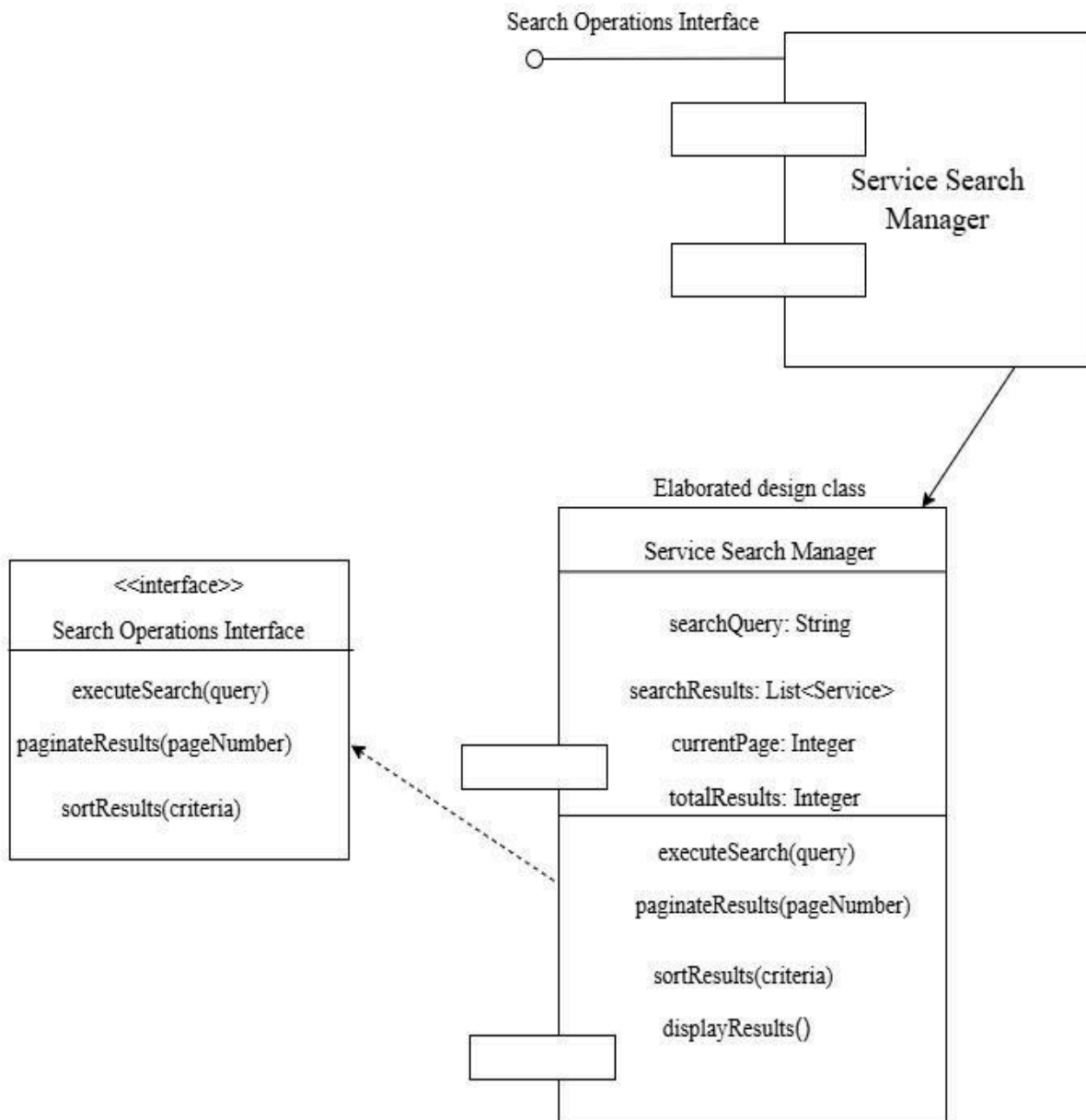


Figure 27: Elaboration of Service Search Manager

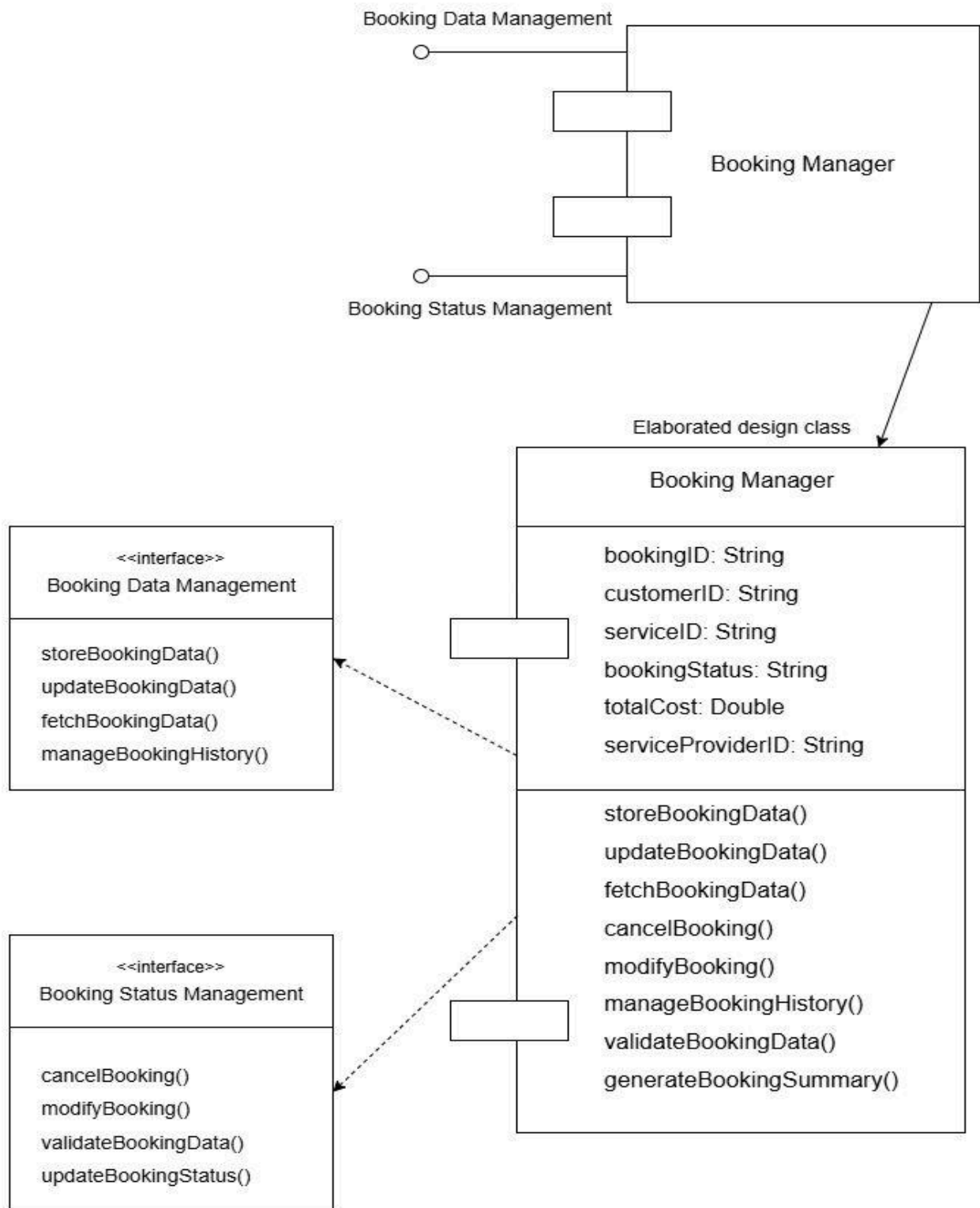


Figure 28: Elaboration of Booking Manager

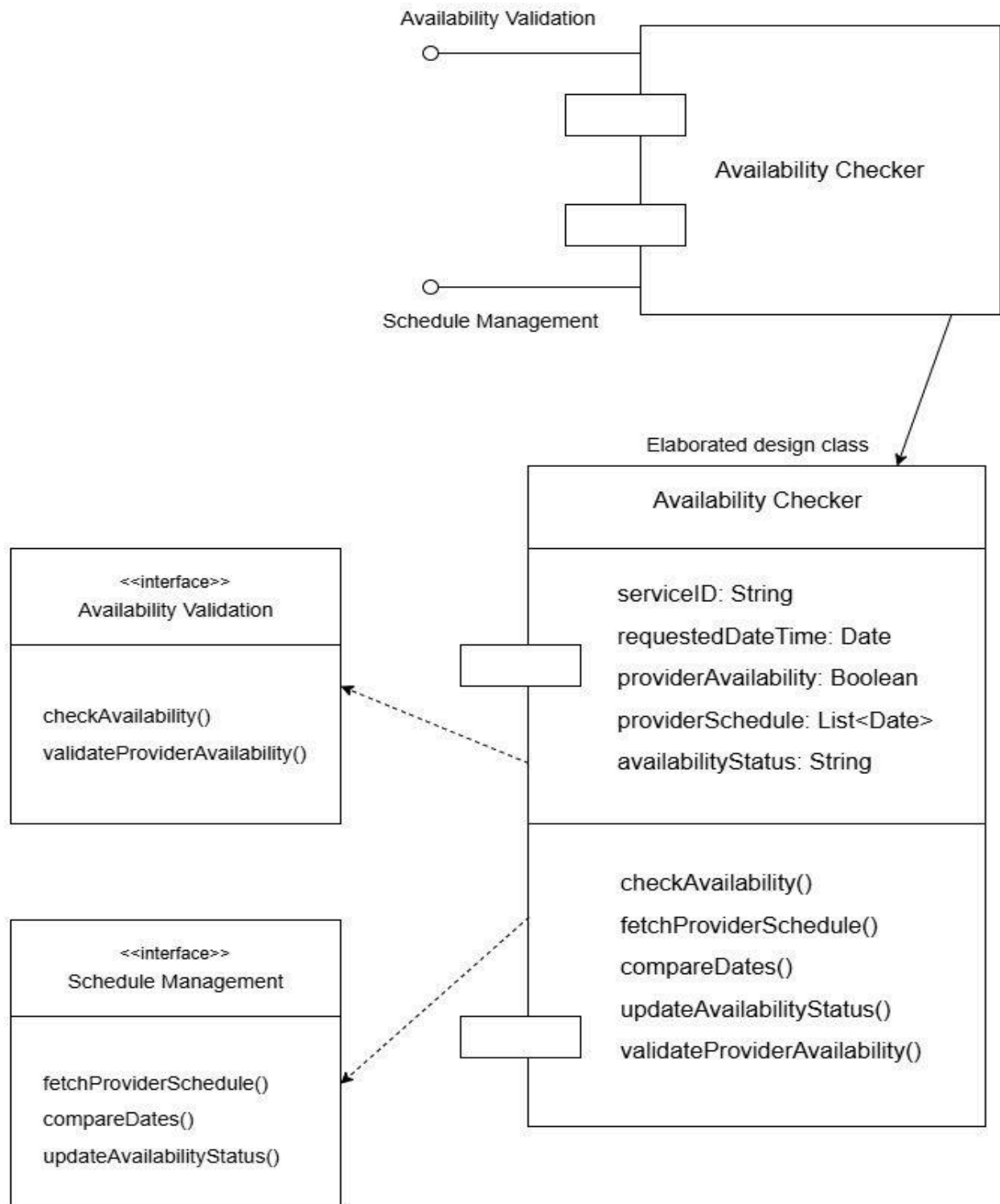


Figure 29: Elaboration of Availability Checker

1.3 Class Diagram

The Class Diagram represents the major software classes derived from the elaborated components. It shows the relationships between entities like Users, Services, Bookings, Payments, and Admin management. It helps to clearly define object-oriented structures, inheritance, associations, and dependencies needed for LocalLinker's development.

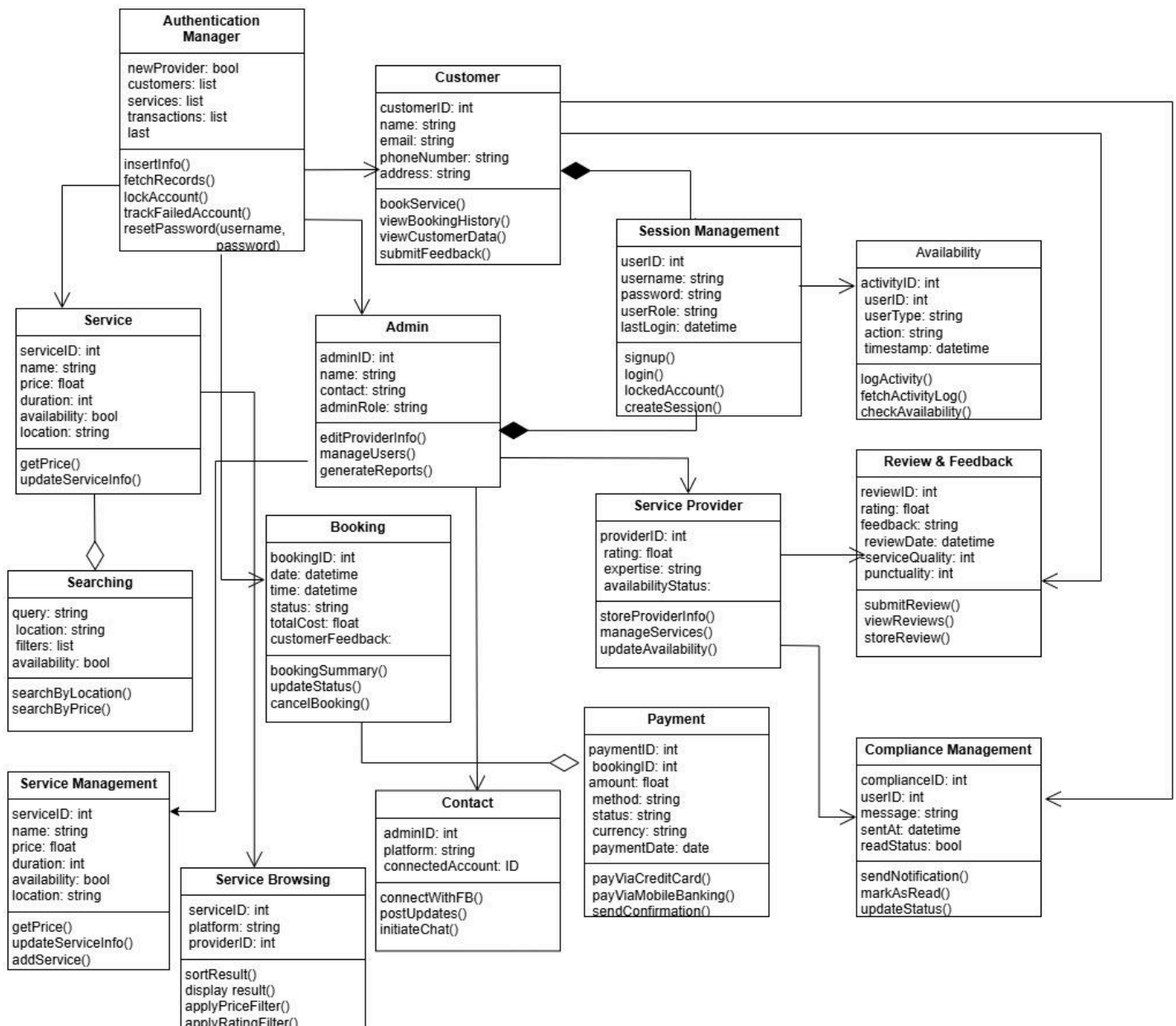


Figure 30:Class Diagram For LocalLinker

2. White Box Testing

White box testing involves examining the internal workings of the software components. In this phase, unit tests will be performed on at least two classes by creating test cases derived from their control flow graphs. This approach ensures that all independent execution paths are thoroughly tested for correctness and robustness.

For the LocalLinker project, we have applied white box testing specifically on two key classes: `registration.php` and `book_service.php`. These classes handle critical functionalities—user registration and service booking respectively—and thorough testing helps guarantee the reliability and accuracy of these components.

For registration.php:

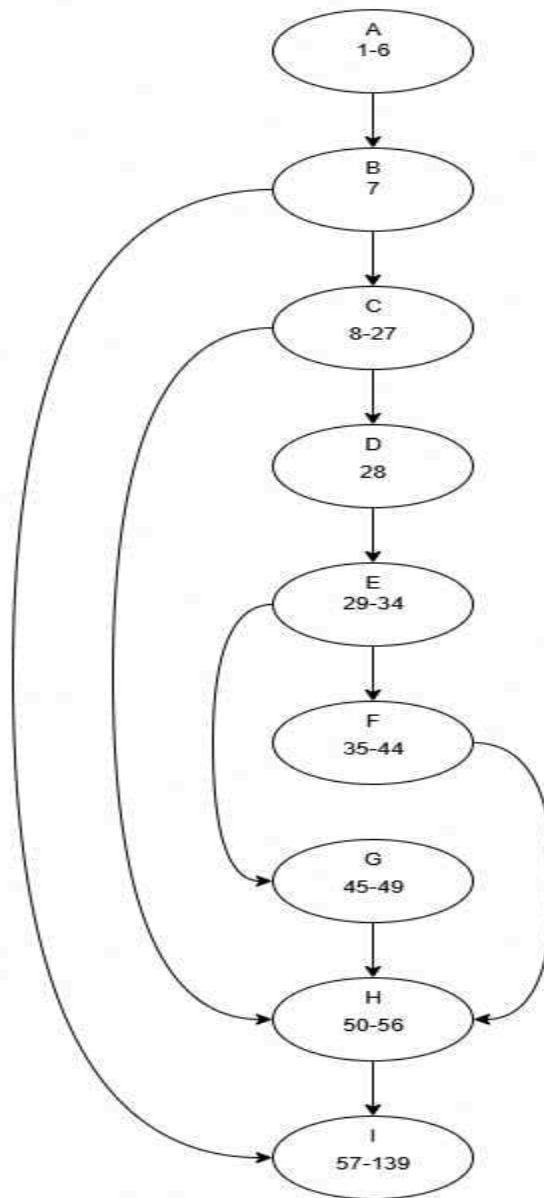


Figure 31:Control Flow Diagram of Registration.php

Cyclomatic complexity Calculation:

Edge = 11

Node = 9

p = 1

Cyclomatic complexity = $e - n + 2p$

= $11 - 9 + 2 \times 1$

= 4

Since the cyclomatic complexity of the graph is 4, there are consequently 4 independent paths within the graph, as shown below:

Independent Paths:

1. A - B - I
2. A - B - C - H - I
3. A - B - C - D - E - G - H - I
4. A - B - C - D - E - F - H - I

Test Case Design from the Independent Paths

Test Case ID	Input Details	Expected Output	Independent Path Covered
TC1	Non-POST request (e.g., GET)	Form displayed without processing; no message shown	$A \rightarrow B \rightarrow G \rightarrow H \rightarrow I$
TC2	POST with valid provider data, but service insert fails	Error message for service insert failure; form redisplayed	$A \rightarrow B \rightarrow C \rightarrow D \rightarrow G \rightarrow H \rightarrow I$
TC3	POST with valid provider and service data; all inserts succeed	Success message shown; form redisplayed	$A \rightarrow B \rightarrow C \rightarrow E \rightarrow G \rightarrow H \rightarrow I$
TC4	POST with invalid provider data causing provider insert fail	Error message for provider insert failure; form redisplayed	$A \rightarrow B \rightarrow C \rightarrow F \rightarrow G \rightarrow H \rightarrow I$

For Book Service.php

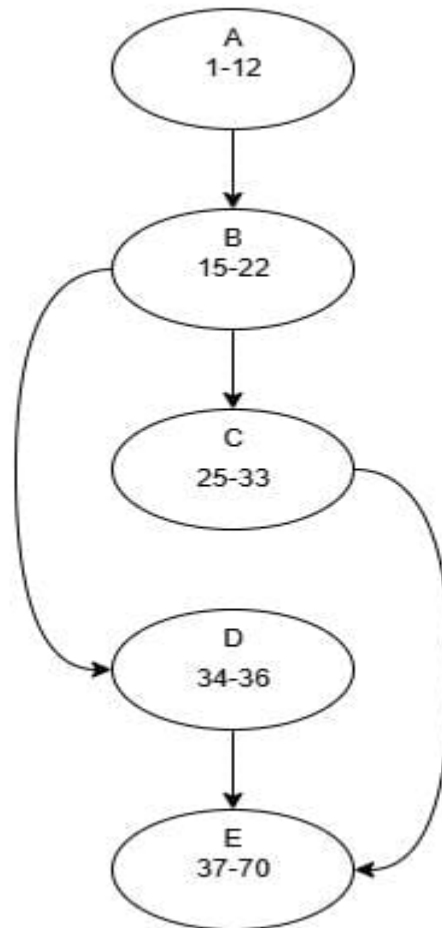


Figure 32:Control Flow Diagram of Book Service.php

Cyclomatic complexity Calculation:

Edge = 5

Node = 5

p = 1

Cyclomatic complexity = $e - n + 2p$

= $5 - 5 + 2 \times 1$

= 2

Since the cyclomatic complexity of the graph is 2, there are consequently 2 independent paths within the graph, as shown below:

Independent Paths:

1. A - B - C - E
2. A - B - D - E

Test Case Design from the Independent Paths

Test Case ID	Input Line	Expected Output	Independent Paths Covered
1	name = "Cleaning"category = "Home"price = 40 quantity = 2 duration = 3 (Then click Confirm)	Booking summary with total \$80 → Form posts to price.php	A→B→C→E
2	name = "Electrician"category = "Repair"price = 60 quantity = 1 duration = 1 (Then click Cancel)	Booking summary with total \$60 → Redirect to search_results.php	A→B→D→E

3. Black Box Testing

Black box testing evaluates the system's external behavior by testing core features against expected outcomes. Functional tests will be designed for at least two critical features using boundary value analysis to verify the system's response to input limits and edge cases, ensuring reliable user-facing performance.

For the LocalLinker project, black box testing is performed on two essential features: login.php and search_results.php. These features are fundamental for user authentication and service discovery, and testing their behavior under various input conditions helps ensure a smooth and error-free experience for end users.

Login feature:

The decision table for the feature is shown below:

		Rule 1	Rule 2	Rule 3
Condition Stub	C1: Email Exists	True	True	False
	C2: Password Matches	True	False	N/A
Action Stub	A1: Redirect to profile	X		
	A2: Invalid password		X	
	A3: User not found			X

The test cases derived from the decision table are given below:

Test Case ID	Email	Password	Expected Outcome
1	testuser@example.com	correct password	Redirect to cprof.php
2	testuser@example.com	incorrect password	Display: "Invalid password"
3	nonexistent@example.com	anypassword	Display: "No user found with that email"

Search_results Feature:

The decision table for the feature is shown below:

		Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8
Condition Stub	C1: Category Provided	True	True	True	True	False	False	False	False
	C2: Location Provided	True	True	True	False	True	False	True	False
	C3: Price Provided	True	True	False	True	True	True	False	False
	C4: Service Availability	True	False						
Action Stub	A1: Show Search Results	X							
	A2: No Services Found		X	X	X	X	X		
	A3: Show Booking Form	X							
	A4: Display Service Unavailable							X	

The test cases derived from the decision table are given below:

Test Case ID	Category	Location	Price	Expected Outcome
1	Cleaning	Dhaka	1	Display search results with the booking option
2	Cleaning	Dhaka	0	Display "No services found matching your criteria"
3	Plumbing	Khulna	9999	Display search results with the booking option
4	Plumbing	Rangpur	1000000	Display "No services found matching your criteria"
5	Electrical	Narayanganj	50	Display search results with the booking option
6	Electrical	Barisal	-1	Display "No services found matching your criteria"
7	Painting	Comilla	500	Display search results with the booking option
8	Painting	Dhaka	999	Display search results with the booking option

4. Bug Detection and Solution

During the testing phase, all test cases executed successfully without any failures. No bugs or defects were identified in the tested components and features of the LocalLinker project. As a result, there was no need for code corrections or rework.

The absence of bugs in the test cases indicates that the implementation meets the specified requirements and behaves as expected under the tested conditions. Continuous testing and monitoring will be maintained to promptly address any issues that may arise in future development or deployment stages.